



CNV DETECTION: EVALUATION OF A MACHINE LEARNING MODEL ACROSS NGS DATA

Beatriz Sánchez Espada

MASTER OF SCIENCE IN BIOINFORMATICS
(2024-2025)

Academic Tutor:

Sònia Casillas Viladerrams

Project Tutor:

Rosa Barcelona Cabeza

Index

1	Introduction.....	1
1.1	Next generation sequencing data for the prediction of CNVs	2
1.1.1	Whole genome sequencing (WGS).....	3
1.1.2	Whole exome sequencing (WES)	3
1.1.3	Targeted sequencing (TS)	3
1.1.4	General methods for the detection of CNVs.....	4
1.2	Machine Learning	6
1.2.1	Supervised learning.....	7
1.2.2	Model development	9
1.3	Machine learning on the prediction of CNVs	9
1.3.1	Applications of Machine Learning for CNV detection.....	11
1.3.2	Challenges for Machine Learning in CNV detection	11
1.4	Context of the project.....	12
2	Objectives	12
3	Methodology	13
3.1	Data Sources	13
3.2	Evaluation metrics	14
3.3	Original model description	15
3.3.1	Metadata extraction pipeline (DeVCopy Pipeline).....	16
3.3.2	Original machine learning model.....	17
3.4	Validation of the original model	18
3.4.1	Changes in the DeVCopy pipeline.....	18

3.4.2	Analysis to improve Model Generalizability	19
3.4.3	Limitations of the original model	19
3.5	Development of a reconstructed model	20
3.5.1	Assessment of the reconstructed model generalizability	21
3.6	Application of Domain Adaptation Techniques	22
3.6.1	Kernel Mean Matching (KMM).....	22
3.6.2	Fine tuning	23
3.7	Methodological Error Analysis.....	23
4	Results.....	24
4.1	Validation of the original model.....	24
4.1.1	Analysis of Model Generalizability	26
4.2	Validation of the reconstructed model.....	27
4.2.1	Assessment of Model Generalizability	29
4.3	Domain Adaptation techniques.....	32
4.4	Feature importance.....	35
4.5	Imbalanced tests.....	37
4.5.1	The reconstructed model validated in imbalanced data.....	37
4.5.2	Model Retraining on Imbalanced Data	38
5	Discussion	38
6	Conclusions.....	43
6.1	Future Directions	44
7	Bibliography	45

Abbreviations

aCGH	Array Comparative Genomic Hybridization
CMA	Chromosomal Microarray Analysis
CNV	Copy Number Variation
FISH	Fluorescence In Situ Hybridization
MLPA	Multiplex Ligation-Dependent Probe Amplification
NGS	Next Generation Sequencing
WES	Whole Exome Sequencing
WGS	Whole Genome Sequencing
TS	Targeted Sequencing
SV	Structural Variation
SNV	Single Nucleotide Variant
Indel	Insertion and Deletion
RD	Read Depth
SR	Split Read
PE	Paired End
DA	De Novo Assembly
ML	Machine Learning
AI	Artificial Intelligence
CNN	Convolutional Neural Network
TP	True Positive
TN	True Negative
FP	False Positive
FN	False Negative
LRR	Log R Ratio
DoC	Depth of Coverage
KMM	Kernel Mean Matching

Abstract

Copy number variants are structural variations larger than 50 bp that play a very important role in susceptibility or resistance to vital and complex diseases such as cancers, obesity and important psychiatric disorders. Although its clinical importance, its detection by laboratory methods is still very expensive, time consuming and many of them have gene-specific limitations. That is why in recent years NGS data is being used to detect these mutations in three different levels: WGS, WES and TS. And although TS has become an important routine technique in clinical settings, its limitations make that many CNV detection tools are still primarily developed and optimized for whole-genome sequencing (WGS) data.

This project evaluates the generalizability and robustness of a machine learning model developed for the detection of CNVs in TS data, originally trained with artificially balanced datasets. Although the original model had good results with a f1-score and accuracy > 0.9 , in this study it is demonstrated that this performance does not generalize to other datasets generated with different probe designs and sequencing strategies. To address these limitations, the model was reconstructed and further evaluated using domain adaptation strategies, such as fine-tuning and kernel mean matching, aiming at improving its ability to generalize across different sequencing conditions.

Although domain adaptation techniques, particularly fine-tuning, improved the model's performance on previously unseen datasets, the results still showed limited generalization to other probe designs and sequencing platforms. Furthermore, experiments with imbalanced data revealed a strong bias toward the majority class, reducing the model's reliability in realistic clinical scenarios. These findings underscore the challenges of developing generalized ML-based CNV detection tools in TS data and highlight the need for improved training strategies and balanced evaluation protocols in biomedical machine learning.

1 Introduction

1 Introduction

The human genome has undergone significant changes across generations due to genetic variability, a process that has facilitated the evolution and adaptation of our species. Upon this variability, there are two main types of variations: structural variations (SVs) and small variants, which include single nucleotide variants (SNVs), and small insertions and deletions (indels) (Zarrei *et al.*, 2015). Structural variations are defined as DNA variations larger than 50 base pairs (bp) while indels affect a segment between 1 to 50 bps and SNVs only 1 bp. SVs are also separated into two types of variants, copy number variants (CNVs), that include deletions and duplications, and complex rearrangements that include insertions, inversions and translocations (Coutelier *et al.*, 2021). Genomic studies have shown that approximately two-thirds of the human genome is based on diverse repetitions, (de Koning *et al.*, 2011) and approximately 4.8-9.5% of the whole genome can be classified as CNVs (Zarrei *et al.*, 2015).

CNVs are normally inherited and so are defined as germline variations, but some arise as *de novo* errors during DNA replication, called somatic CNVs. These two types of variations play a very important role in susceptibility or resistance to vital and complex diseases such as cancers (Ronemus *et al.*, 2025), obesity (Phillips *et al.*, 2020) and important psychiatric disorders such as autism, (Pinto *et al.*, 2014), schizophrenia (Malhotra & Sebat, 2012) or Alzheimer (Guffanti *et al.*, 2013).

Unlike somatic CNVs, which are acquired during tumorigenesis, germline CNVs are present from birth and can disrupt gene function or dosage, leading to developmental abnormalities or increased disease susceptibility (Curtis *et al.*, 2012). For example, deletions in BRCA1 are associated with hereditary breast and ovarian cancer syndromes (Hakkaart *et al.*, 2022), while alterations in TP53 can cause Li-Fraumeni syndrome (Guha & Malkin, 2017), both significantly increasing lifetime cancer risk. Germline CNVs in genes such as APC (familial adenomatous polyposis) or MSH2 (Lynch syndrome) similarly predispose individuals to specific diseases (Agiannitopoulos, 2023; Alcantara *et al.*, 2022). In addition to cancer, recurrent germline CNVs like the 22q11.2 deletion (DiGeorge syndrome) (Hiroi *et al.*, 2013) or the 16p11.2 duplication (Posar & Visconti, 2020) are linked to neurodevelopmental

1 Introduction

disorders such as intellectual disability and autism spectrum disorders. Rapid and accurate detection of germline CNVs is essential for early diagnosis, personalized surveillance strategies, and informed reproductive counseling, thereby improving clinical outcomes for affected individuals and families (Xie *et al.*, 2024).

Many methods and technologies have been used for the detection of CNVs in clinical diagnosis settings. The most commonly used are multiplex ligation-dependent probe amplification (MLPA), chromosomal microarray analysis (CMA), array comparative genomic hybridization (aCGH) or fluorescence *in situ* hybridization (FISH) (Liu *et al.*, 2020). MLPA is limited by the number of probes in the kit, approximately 50 probes, so it is almost only used for testing targeted data. Chromosomal microarray analysis (CMA) is currently the standard method for detecting copy number variations. However, its resolution is limited, as clinically significant findings are generally reported only for variations larger than 200 kilobases (Whitford *et al.*, 2019). FISH, although it can reach a higher resolution, it is still not enough as only large repeats such as whole gene repeats can be detected (Liu *et al.*, 2020). Finally, while the aCGH method is more sensitive, it has an important problem as it is limited to only detect CNVs of sequences present in the reference assembly used in the array probes (Singh *et al.*, 2021). In general, these methods are still very expensive, time consuming and many of them have gene-specific limitations (Munté *et al.*, 2024).

More recently, next-generation sequencing (NGS) is beginning to be used as a first-line screening tool for CNVs, especially in combination with SNV detection. Although it is not yet the clinical gold standard, its broader genomic coverage and potential for simultaneous variant detection make it a promising alternative for complete genetic diagnosis (Riggs *et al.*, 2020).

1.1 Next generation sequencing data for the prediction of CNVs

With the development of next-generation sequencing (NGS) in the past decade, this technology has become the standard for general genomic analysis, as it is able to sequence millions of fragments simultaneously and is cost-effective. NGS is currently used to detect multiple types of genetic variations, including CNVs. In addition, through NGS we can sequence at three different levels of the genome: whole genome level, whole exome or a panel-based targeted manner (Neveu *et al.*, 2024).

1 Introduction

Nevertheless, while whole-genome sequencing (WGS) is not constrained by probe design, whole-exome sequencing (WES) and targeted sequencing (TS) are influenced by the design and performance of capture probes, which can impact coverage and variant detection. Overall, NGS offers significantly improved sensitivity and resolution for the accurate detection of genomic variants (Povysil *et al.*, 2017).

1.1.1 Whole genome sequencing (WGS)

WGS provides genome-wide coverage resulting in the detection of CNVs in coding and non-coding regions. This approach has some advantages, as it offers high sensitivity and accuracy due to an uniform coverage of the genome and is also able to detect CNVs in intronic and intergenic regions (Francisco De La Vega *et al.*, 2024). Nevertheless, this method has two main disadvantages. In first place WGS remains significantly more expensive than targeted or exome sequencing, in terms of laboratory costs and in computational infrastructure and bioinformatic expertise. Also, from a clinical perspective many variants detected outside coding regions are not clinically significant as most of the disease-causing CNVs span in exons (Meienberg *et al.*, 2016).

1.1.2 Whole exome sequencing (WES)

On the other hand, whole-exome sequencing (WES) targets all protein-coding regions (exons), achieving higher sequencing depth in these regions compared to whole-genome sequencing (WGS) and theoretically improving the ability to detect copy number variants (CNVs) affecting coding sequences (Burdick *et al.*, 2020). This approach provides a cost-effective and clinically relevant alternative to whole-genome sequencing (WGS), but it has its own limitations. Despite the increased depth, WES often provides incomplete and uneven coverage across exons which frequently results in missed or poorly covered exonic regions. This non-uniformity significantly complicates the detection of CNVs, particularly in smaller or single-exon events, as the variability in coverage makes the detection more challenging. Accurate detection of such variants typically requires specialized algorithms and high-quality, contiguous sequencing data that WES may not provide (Gordeeva *et al.*, 2021)

1.1.3 Targeted sequencing (TS)

1 Introduction

WES and WGS are very useful in research fields but for clinical settings, targeted sequencing has become an important routine technique. TS focuses on a particular cluster of genomic regions and it enables easier data interpretation thanks to its high coverage depth. It also offers reduced costs and quicker processing, key advantages for many industrial and clinical applications where time and budget are critical factors (Pei *et al.*, 2023). However, despite these advantages, identifying CNVs in targeted data remains challenging because of many factors, such as DNA quality and quantity, target enrichment through hybridization in NGS, fragment lengths, GC content, and PCR amplification during library preparation. These factors can lead to variations in the representation of a target region within a sample and if these variations differ in quality from other samples, they may result in over- or underrepresentation, which could be misinterpreted as a CNV (Adolfsson *et al.*, 2023)

1.1.4 General methods for the detection of CNVs

There are four main bioinformatic strategies for the detection of CNVs in NGS data: split read (SR), paired end (PE), de novo assembly (DA) and read depth (RD).

The read-pair methods were where the utility of NGS in the detection of CNVs was first demonstrated. This method compares the insert size between the actual sequenced read-pairs with the expected size based on a reference genome. The discordance in distances between mapped paired-reads and the predetermined average insert size is utilized to identify CNVs (Pirooznia *et al.*, 2015).

The split read method uses reads from paired end sequencing where one read has a confident mapping. The unmapped reads can indicate a source of breakpoint at a single base pair level and when these are mapped can provide the start-end positions of indels (Pirooznia *et al.*, 2015).

De novo assembly methods rely on the theoretical premise that all forms of genetic variation can be detected by the assembly of short reads if they are accurate enough. The approach is simple, generate contigs that are then compared with the reference genome. However this method is not commonly used in CNV detection due to its overwhelming computation demand (Pirooznia *et al.*, 2015).

1 Introduction

Read-depth (RD) based algorithms are the most common approach among all the CNVs callers (Li *et al.*, 2024). This method assumes that the read depth in a genomic region is directly proportional to its copy number, meaning it relies on region-specific coverage information (coverage depth), as illustrated in *Figure 1*. RD-based methods are particularly good for targeted gene panels, where deep sequencing provides high coverage depth and thereby increases the sensitivity for detecting CNVs within the covered regions (Singh *et al.*, 2021).

However, due to the limitations of targeted sequencing, such as uneven coverage, lack of reference regions, and dependency on panel design, many CNV detection tools are still primarily developed and optimized for whole-genome sequencing (WGS) data (Zhang *et al.*, 2019). As a result, WGS remains the preferred input for many CNV callers, while targeted sequencing still poses technical challenges for robust CNV detection in clinical applications.

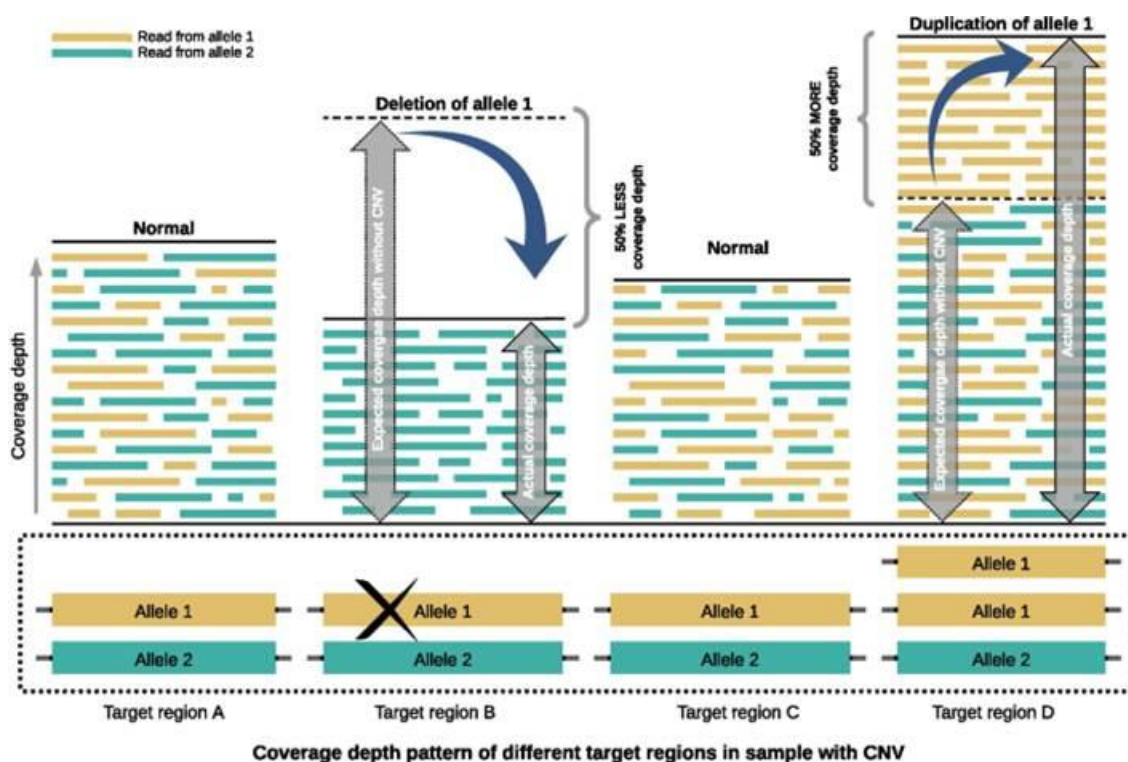


Figure 1. Read depth method for copy number detection. This method relies on changes across different target regions of a sample. The normal coverage depth varies between regions, as seen in regions A and C, making it important to have reference data for each region's expected coverage. In the case of a deletion, such as in region B, the loss of one allele reduces coverage to 50% of the expected depth. In the same way, in region

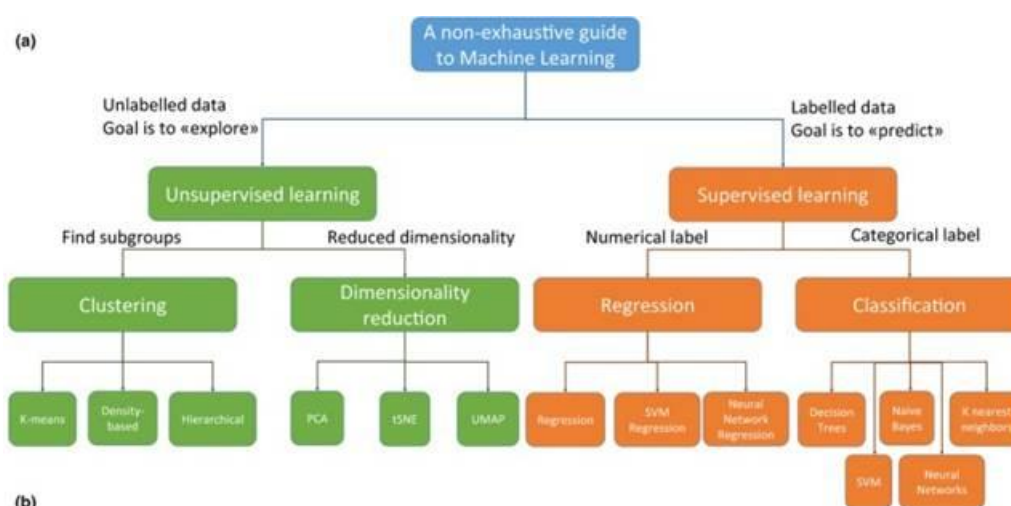
1 Introduction

D, a duplication of one allele increases coverage by 50%, reaching 1.5 times the normal coverage. (Singh *et al.*, 2021)

The lack of standardized guidelines across the wide range of next-generation sequencing (NGS) platforms and methodologies combined with the absence of a gold standard for evaluating these algorithms, leads to significant variability in the performance of different detection methods (Xie *et al.*, 2024). This variability underscores the urgent need for researchers to develop effective and reliable strategies for CNVs detection in targeted sequencing data (Pei *et al.*, 2023).

1.2 Machine Learning

Machine learning (ML) is a subfield of artificial intelligence (AI) that can give systems the ability to learn without being explicitly programmed to do so. This field is focused on developing and improving algorithms and statistical models to learn from certain data and improve its performance on a specific task. ML algorithms identify patterns and relationships in large datasets and build models to make predictions and decisions based on new input data (Gowri *et al.*, 2019). In machine learning there are two main types of approaches, unsupervised learning and supervised learning (*Figure 2*). The primary difference between these approaches lies in the use of labeled data: supervised learning requires labeled datasets, while unsupervised learning operates on unlabeled data (Badillo *et al.*, 2020).



1 Introduction

Figure 2. Overview of the main machine learning (ML) algorithms. The overview is split in supervised and unsupervised learning showing the most common algorithms of each approach depending on the methodology used. (Badillo *et al.*, 2020)

1.2.1 Unsupervised learning

If the labels are not known or if the objective is to examine the natural patterns of the data, unsupervised learning must be used. For biological and clinical applications, the most commonly used methods in this approach are clustering and dimensionality reduction (Badillo *et al.*, 2020).

In the first one the main objective is to determine relevant subgroups in a given dataset and understand the structure of the data without knowing the specific properties of these clusters. This method is used in many ways in genetic contexts, as transcriptomic, metabolomic or proteomic datasets may have many features within the context of tens of thousands of measures per sample (Badillo *et al.*, 2020).

In the second one the aim is to project high-dimensional data into two or more dimensions while maintaining the variability and the relative distances of the data. Normally principal component analysis is widely used in biological applications, but in the terms of machine learning more recent and advanced methods are being developed (Badillo *et al.*, 2020). Some examples are autoencoders for neural network-based approaches or t-Distributed Stochastic Neighbour Embedding (Nguyen & Holmes, 2019).

1.2.1 Supervised learning

In supervised learning, training data is provided along with corresponding labels. The goal is to learn the rules of this association and create a model that will be able to predict the output for new unseen input data. In this case it also has two main categories: Classification and regression (Badillo *et al.*, 2020).

Regression models aim to predict continuous values or numeric outputs, for example a prediction of the days a patient may stay in an intensive care unit. The more common regression algorithms used in biomedical fields are linear regressions, decision trees or neural networks. Linear regressions are normally used for its simplicity and interpretability in ordinary tasks,

1 Introduction

but nowadays other machine learning models are more used for its predictive power (Pantanowitz *et al.*, 2025).

The objective of classification models is to predict categorical labels or classes such a tumor versus a healthy tissue. In this case the most used algorithms are random forest (a decision tree method) and CNNs (convolutional neural networks). In some cases the same methods can be used in both regression and classification models (Pantanowitz *et al.*, 2025). Classification methods in supervised learning are proved to be the most used algorithms in the biomedical field and it is the type of machine learning approach related to this project, so it is going to be explained in more detail, specially for neural networks

1.2.1.1 Decision trees

Decision trees have become very popular methods to build machine learning models because of its ability to predict patients outcomes, diagnose diseases and even identify new biomarkers (Cheung *et al.*, 2023). In addition, in comparison with other methods, decision trees are simple and easy to visualize. Nevertheless, currently decision trees in its basic form are almost never used because they have a high propensity to cause overfitting, especially with biological data that tends to be highly dimensional. Anyways, decision trees are quite used in other two approaches: gradient boosting and random forest.

Random forests work by constructing hundreds of deep decision trees and combining all the outputs to avoid overfitting. Gradient boosting does the contrary, each tree is a shallow decision tree and the algorithm decreases the error by adding more trees over time (Badillo *et al.*, 2020). This approach helps in minimizing the overfitting and the instability that are typical of simple decision trees, nevertheless these problems are very common in all machine learning programs for biomedical data and points to the importance of proper hyperparameter tuning and data preprocessing (Pantanowitz *et al.*, 2025).

1.2.1.2 Neural Networks

Neural Networks are a recent advanced tool in supervised ML that have been very used for AI in medicine due to its efficiency in the high dimensional nature of biological data. This technology is based on the complexity of our brain, as it is composed of layers of interconnected nodes simulating artificial neurons. A typical neural network works with a first

1 Introduction

input layer of neurons that receives the input data and passes that information to the number of hidden layers, performing a series of complex computations and transformations. Finally, it reaches the final output layer where it generates the prediction (Pantanowitz *et al.*, 2025).

In a practical scenario all these methods and technologies work through a model that must be developed and validated to a series of new unseen data to be able to use it in a clinical scenario.

1.2.2 Model development

For the selection or development of a machine learning model, it is first necessary to obtain a high-quality dataset. Once available, the dataset should be divided into three main subsets: training, testing, and validation. Typically, the dataset is split with approximately 60-70% for training, 10-20% for validation, and 20-30% for testing, although these proportions may vary depending on the dataset size and the specific problem (Badillo *et al.*, 2020).

The training set is used to build the model, and, if necessary, the validation set is employed for tuning hyperparameters. Then, the model with the best performance in the validation sets is chosen for the posterior analysis. To evaluate generalization error, the test set is used to measure prediction error on data not involved in training. Finally, in order to validate and assess the model's generalizability, testing should be performed on a fully independent dataset (Badillo *et al.*, 2020).

1.3 Machine learning on the prediction of CNVs

The application of machine learning (ML) to the prediction of copy number variants (CNVs) has emerged as a promising method to overcome the limitations of traditional CNV detection tools, particularly in the context of targeted sequencing data (Hill & Unckless, 2019).

Traditional CNV callers, such asXHMM, ExomeDepth (Plagnol *et al.*, 2012), and CODEX (Jiang *et al.*, 2015), were originally developed for whole-exome or whole-genome sequencing and are very dependent on consistent and uniform read-depth coverage. When applied to targeted gene panels, these methods often suffer from reduced sensitivity and specificity due to the variability in capture efficiency, GC-content biases, and the high degree

1 Introduction

of customization required for each panel. In clinical settings, where targeted sequencing is frequently employed for its cost-effectiveness and deep coverage, these limitations become particularly problematic (Povysil *et al.*, 2017; Zhang *et al.*, 2019)

A recent benchmarking study on CNV callers for targeted data was conducted by Munté using 12 tools for CNV detection. In this study, the most efficient and accurate tools were GATK-gCNV and ClinCNV (Munté *et al.*, 2024). Despite their high reported accuracy and F1-score, a major limitation of these tools is that adapting them to different gene panels often requires changes in methodology, tuning of hyperparameters, and retraining of models for each new dataset. This is because these types of models are highly dependent on the coverage profile, which varies significantly between kits and sequencing platforms.

Another CNV caller more focused on targeted data is DECoN (Fowler *et al.*, 2016), an improvement of ExomeDepth (Plagnol *et al.*, 2012). DECoN presents a more adaptable option for CNV detection in targeted sequencing data. It employs a reference-based normalization strategy that relies on control samples to account for coverage variability, making it less dependent on panel-specific training and more robust across different sequencing platforms (Fowler *et al.*, 2016). Similarly, isoCNV is designed to optimize parameters using in silico validated data, increasing the sensitivity of other CNV callers in TS data (Barcelona-Cabeza *et al.*, 2021).

Nevertheless, the dependency on reference samples or large batches of samples for the proper functioning of the software tools is an important limitation. For instance, GATK-gCNV offers a case mode that allows the analysis of a single sample, however, it requires a pre-constructed model trained on a cohort of samples (Babadi *et al.*, 2023). ClinCNV is designed primarily as a multi-sample caller, requiring a minimum of 10 samples, with optimal performance achieved with around 40 samples (Demidov *et al.*, 2022). Similarly, both DECoN and ExomeDepth always rely on a batch of reference samples for accurate analysis. In addition, the challenge of having sufficient high-quality data from the same sequencing technology and under identical experimental conditions is not only technically demanding but also extremely expensive (Schwarze *et al.*, 2020). This highlights the need for a robust tool capable of accurately detecting copy number variations (CNVs) from single-sample targeted sequencing data, as no currently available software can achieve high accuracy under these conditions.

1 Introduction

1.3.1 Applications of Machine Learning for CNV detection

In response to these challenges, researchers have used machine learning approaches to improve CNV detection. One of the first works in this domain by Pounraja in 2019 (Pounraja *et al.*, 2019) introduced a supervised learning method that utilized a random forest classifier trained on features such as normalized read depth, GC content, and exon length. Although the model showed improvements in accuracy over traditional tools, it was totally dependent on the output of existing CNV callers for its input features, thereby functioning as a classifier rather than an independent CNV detection method.

More recent approaches have shifted toward deep learning architectures, with convolutional neural networks (CNNs) being particularly prominent. A representative example is DeepCNV, a CNN-based model developed by Glessner (Glessner *et al.*, 2021), which detects CNVs directly from raw read depth data without requiring manual feature engineering. This study demonstrates the ability of deep learning models to capture complex patterns in sequencing data and significantly reduce the need for handcrafted input features. However, DeepCNV was primarily trained and validated using whole-genome sequencing data and demands considerable computational resources and large-scale labeled datasets, which limits its current applicability in clinical contexts where targeted sequencing is more common.

1.3.2 Challenges for Machine Learning in CNV detection

Despite the innovations shown by ML and deep learning approaches, several challenges remain. One important limitation is the lack of high-quality, labeled datasets for training and validation, particularly for rare CNVs and targeted panels. Additionally, many existing ML models are not designed to generalize across different panel architectures or sequencing technologies. This often results in the need for retraining or fine-tuning on a panel-specific basis, which in a certain way, replicates the limitations of traditional CNV callers they were intended to improve. Another issue is that most published studies do not include cross-panel validation, raising questions about the robustness and generality of their models in real-world clinical data.

Given these limitations, there is an important need for machine learning models that are not specific to a single panel, robust across sequencing platforms, and capable of operating

2 Objectives

independently from traditional CNV detection tools. This project aims to address this gap by validating and improving a supervised ML model that prioritizes cross-panel generalizability and reduced dependency on panel-specific features. By proving the model on diverse datasets and evaluating its performance across multiple gene panels, the goal is to contribute a scalable and clinically viable solution for CNV detection in targeted sequencing data.

1.4 Context of the project

This thesis builds upon a previous study conducted by (Doorn, 2022), which compared two machine learning models (TabNet and XGBoost) for the detection of copy number variants (CNVs) implementing a metadata extraction pipeline to avoid the need for a batch of reference samples. In this study two different models were generated from two gene panel datasets: ICR96 and a Twist-generated dataset. TabNet, a deep learning model designed for tabular data, was chosen for its ability to handle complex feature interactions and provide interpretable results, offering advantages over traditional gradient boosting methods like XGBoost. The study demonstrated that the TabNet model achieved high accuracy (0.98) in the prediction of regions containing CNVs (Doorn, 2022).

Based on these findings, the initial goals of this thesis were to validate the promising results of the TabNet model (hereafter referred to as the original model) on the Twist dataset and to improve the model performance on other targeted sequencing panels such as ICR96. Additionally, there was an interest in exploring the possibility of extending this approach to broader sequencing strategies, including whole exome sequencing (WES) and whole genome sequencing (WGS). To achieve these objectives, this research aimed to analyse the validation results and investigate advanced machine learning techniques such as transfer learning and domain adaptation.

2 Objectives

These results underscore the need for CNV detection models that are both robust and generalizable across probe designs, without the need of training a new model with a batch of samples for each analysis. While early findings were encouraging, as the research progressed,

3 Methodology

it became clear that the original model had significant methodological limitations and challenges that needed to be addressed first.

This work aims to address the limitations of the original CNV detection model and improve its performance through a more robust and generalizable approach. The following specific objectives will guide this process:

1. Analyze the reproducibility of the original model to evaluate the consistency and reliability of its results.
2. Validate the original model across diverse sequencing platforms and data types by assessing performance on datasets generated with different probe designs (e.g., Twist, Agilent) and data types (e.g., targeted gene panels, whole-exome sequencing).
3. Apply domain adaptation techniques to enhance model accuracy and robustness when used on varying sequencing technologies and data types.
4. Identify and address methodological biases, including class imbalance and artificial balancing during validation, by assessing their impact on model training and evaluation and proposing effective mitigation strategies to improve predictive performance.

3 Methodology

3.1 Data Sources

To evaluate the model effectively, a diverse set of datasets representing various sequencing technologies and probe design were used. Specifically, the evaluation included two targeted sequencing (TS) datasets from Twist Bioscience, one TS dataset from Illumina, and a whole-exome sequencing (WES) dataset from Roche NimbleGen.

Two internal datasets based on Twist Bioscience technology (batches 202502 and 202503) assessing across 1075 genes were included. As these lacked validated CNVs, variants were inferred using the DECoN pipeline with IsoCNV for the improvement of its parameters (Fowler *et al.*, 2016).

3 Methodology

The ICR96 Exon CNV Validation Series was also used. The dataset includes high-quality sequencing data using Illumina HiSeq 2500, from the targeted NGS assay: TruSight Cancer Panel. This panel assesses germline mutation across 113 genes and in this series includes MLPA results in 96 independent samples for CNV detection. Including 66 samples containing at least one validated exon CNV and 30 samples with validated negative results for exon CNVs in 26 genes (Mahamdallie *et al.*, 2017).

Finally, whole-exome sequencing (WES) data from the Roche NimbleGen platform was incorporated. This dataset comprises 33 samples across 19033 genes with a laboratory CNV validation using SNP microarray (Sanders *et al.*, 2012). This dataset is going to be posteriorly refereed in all the study as “*NG dataset*”.

The original model was trained using a dataset with the same characteristics as the two Twist datasets that are used in this study for validation. In addition, the ICR96 dataset, which originally had bad results, and WES data from NimbleGen were also used to address generability.

3.2 Evaluation metrics

For the correct evaluation of the model and its validation in the different datasets, different evaluation metrics were used in this project. The metrics used are accuracy, recall, precision and F1-score, and are calculated as follows (Edwin Aco Tito *et al.*, n.d.) :

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall/Sensitivity = \frac{TP}{TP + FN}$$

$$Specificity = \frac{TN}{FP + TN}$$

$$F1 = \frac{2 * TP}{2 * TP + FP + FN}$$

3 Methodology

Where true positives (TP) are CNVs that are correctly predicted by the model, meaning the model identifies a CNV in a region that is also annotated as such. TN are true negatives, normal regions that are predicted as normal. False positives (FP) refer to normal regions incorrectly predicted to contain CNVs, while false negatives (FN) are annotated CNV regions that the model fails to detect, classifying them instead as normal.

Specifically, false negatives are very important in clinical and diagnostic settings, as missing a true CNV can lead to misdiagnosis, delayed treatment, or failure to detect a disease-causing variant.

Among the available evaluation metrics, this study focuses on accuracy, recall, precision, and the F1-score as key indicators of model performance.

3.3 Original model description

To validate the initial model, the first step is to follow the metadata extraction pipeline developed by Doorn in his master's thesis (TFM) (Doorn, R. B., 2022). With the objective of not needing a batch of samples for the prediction, his pipeline is designed to extract various types of metadata from a given NGS dataset, aggregate the information per genomic region, and merge it into a data frame. This data frame is then used as input to train a machine learning model focused on the detection of copy number variations (CNVs). All the process is summarized in *Figure 3*.

3 Methodology

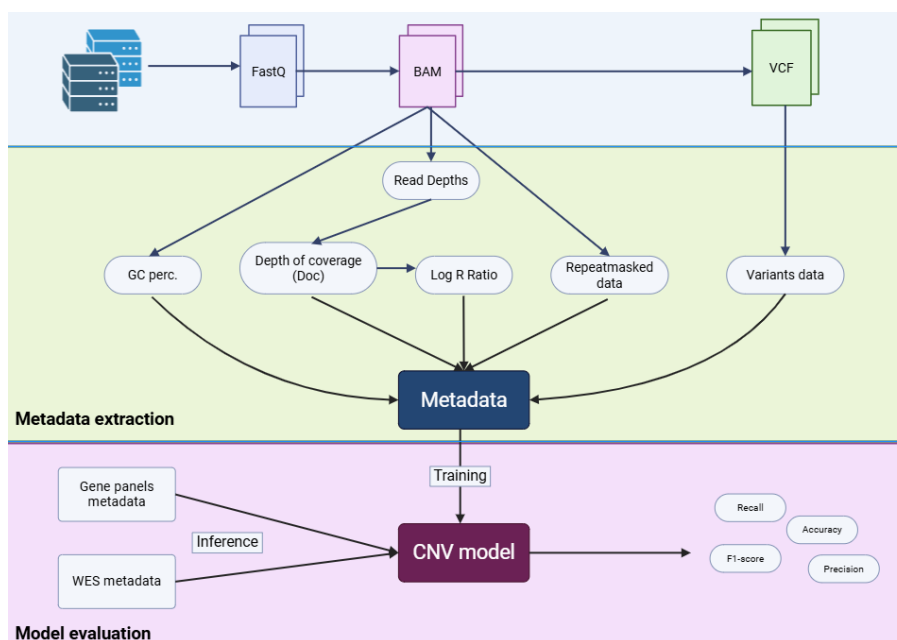


Figure 3. Workflow of CNV model evaluation. The pipeline starts from raw sequencing data (FastQ), which is aligned to produce BAM and VCF files. From these, relevant features are extracted: GC content, depth of coverage (DoC), Log R Ratio, repeat-masked regions, and variant data. These are compiled into a metadata table used to train a machine learning model for CNV prediction. The model is then evaluated using metadata from different sequencing strategies (gene panels, WES), and performance is assessed using metrics such as recall, precision, F1-score, and accuracy.

3.3.1 Metadata extraction pipeline (DeVCopy Pipeline)

The pipeline starts by extracting the GC content for each region using the bedtools nuc tool (Bedtools 2.30.0, n.d.). Next, repeat-masked information is collected by intersecting the regions of interest with the RepeatMasker annotation (Repeatmasker 4.0.5, n.d.) in the hg19 reference genome, using bedtools intersect (Bedtools 2.30.0, n.d.).

The third type of metadata extracted is the number of heterozygous and homozygous variants per individual within each region. These are calculated using the vcfR package (1.15.0) in R (Knaus & Grunwald, 2023), which allows parsing and processing of VCF (Variant Call Format) files. Variant density can be indicative of structural variations and may contribute valuable information to the model.

The most important metadata for CNV detection, however, is the read depth. This is computed using sambamba depth base (Sambamba 0.6.6, n.d.), which calculates the

3 Methodology

base-level read coverage across all regions of interest for each individual. Once the raw read depth is obtained, it undergoes a normalization process using an R script. This script first adjusts each base's depth by dividing it by the mean depth of the individual, correcting for inter-sample variation in sequencing depth. Then, to prevent issues in the posterior log2 ratio calculations (where values close to or equal to zero are problematic), all normalized values are shifted by adding the absolute minimum depth across the sample plus one, ensuring a minimum value of one. Finally, the script calculates the mean depth of coverage (doc) and its standard deviation (doc.sd) for each region.

Once all the metadata is extracted, it is merged in a table along with the information of each region, as the start-end positions and length. Then, the files with the validated CNVs from each dataset are used to annotate each region. With this process it is created a table per sample, so all the information is unified for making the posterior steps more simple and less computationally demanding.

Once all information is merged the Log R Ratio (LRR) is calculated. To mitigate class imbalance, a downsampling approach was applied to normal regions, ensuring a more balanced distribution between CNV and non-CNV regions. The LRR is defined as the base-2 logarithm of the ratio between the observed depth of coverage and an expected reference value, calculated as following:

$$LRR = \log_2 (R_Observed / R_Expected)$$

The observed coverage corresponds to the raw sequencing depth for each region, while the expected coverage is estimated using the same method as CNVkit, a sliding window median with a window size of 25, trimmed to reduce the influence of outliers.

With all the metadata merged into a single file, predictions can be made, and evaluation metrics calculated to assess the model's ability to accurately detect CNVs.

3.3.2 Original machine learning model

As said in Section 1.4, the original study compared two machine learning models of different nature. The first one, XGBoost, had a significantly lower performance and was excluded from this study. The second one was Tabnet, a deep learning architecture designed

3 Methodology

specifically for tabular data. The model was trained on 80% of the training data for 250 epochs with a patience of 50. Twenty percent of the data was used as a validation set. The model trained with ICR96 had poor performance results with a F1-score of 0.22, in the case of the model trained with Twist the results were very good with a F1-score of 0.98 (Doorn, R. B., 2022). These results were the start point for the current project of the thesis.

It is important to note that downsampling to balance the class distribution in the training of the model was applied only to the Twist dataset, but not to the ICR96 dataset. This represents the primary distinction between the two datasets and is likely one of the reasons for the differences in model performance. Based on the results of the previous analysis, we first decided to evaluate the model's generalization capability using balanced datasets to determine whether it performs well across datasets with uniform class distribution. Once this was established, we then addressed the issue of class imbalance.

3.4 Validation of the original model

3.4.1 Changes in the DeVCopy pipeline

To evaluate the generalization capacity of the original model, it was applied directly to the datasets commented in the Section 3.1, without modifying the model itself, as an initial testing strategy. However, several scripts and commands had to be adapted to accommodate the different formats of these new datasets.

One of the main adjustments involved the CNV file used to annotate the metadata table and calculation of metrics. In the original pipeline, this file was generated using DECoN, a widely used CNV caller that improves upon the well-established ExomeDepth. More specifically, the file was produced using isoCNV (Barcelona-Cabeza *et al.*, 2021), a tool developed by Rosa Barcelona to optimize DECoN's hyperparameters and enhance its accuracy. For robust validation of the model, however, CNV files clinically validated through laboratory techniques such as MLPA were also incorporated. These validated CNV files differed in format from those produced by DECoN, so a custom Bash script was developed to standardize all CNV files into a common format, ensuring compatibility with the pipeline.

3 Methodology

Additional modifications were made to reduce the computational load of the pipeline on the servers used across projects. While the original pipeline was designed for smaller datasets such as targeted gene panels, which do not pose significant computational challenges, the inclusion of whole exome datasets, often hundreds of gigabytes in size, introduced substantial demands on system resources. As a result, scripts responsible for read depth normalization (used in the calculation of depth of coverage) and metadata table generation were optimized to reduce memory usage and enable efficient execution on limited-resource servers.

3.4.2 Analysis to improve Model Generalizability

Following the initial validation, two additional tests were conducted to investigate the reduced model performance observed on the ICR96 and WES datasets.

In contrast to the Twist datasets, where validated CNVs were identified using DECoN and IsoCNV, the ICR96 and WES datasets relied on laboratory-validated CNV annotations. To determine whether this methodological difference contributed to the performance difference, CNVs were also called from the BAM files of both datasets using the same pipeline applied to the Twist data. This process generated BED files with regions labeled as Normal, DUP, or DEL based on the DECoN output.

A second test was performed to evaluate whether filtering both datasets to retain only the intersect regions with those seen during training could affect model performance. This filtering was performed using bedtools intersect (Bedtools 2.30.0, n.d.), comparing the target file from the batch 202502, with the same nature of the training data, and the target file of the dataset to be filtered. It is also important to address that this analysis uses the MLPA validated CNVs in order to compare the filter efficiency with the original performance.

3.4.3 Limitations of the original model

After the validation of the initial model, it was decided to use transfer learning approaches to improve its predictive performance and generalization capabilities. Transfer learning has demonstrated considerable effectiveness in cases where labeled data are limited or when reusing knowledge from pre-trained models can improve its accuracy (de

3 Methodology

Mathelin *et al.*, 2023). Nevertheless, in order to apply these approaches to the initial model we needed to access the model's internal architecture and trainable parameters but these had important challenges for further development.

The original TabNet model had been exported using Vertex AI and called through a prediction server in a Docker container. This configuration, while suitable for inference tasks, is difficult for further improvement due to the format in which the model was saved. Specifically, the model was stored using TensorFlow's "SavedModel" format through the `tf.saved_model.load(path)` function. Although this format is efficient for model serving and inference, it is very restrictive for training and doing architectural modifications, as it does not readily expose the model's structure in a manner compatible with the `fit()` function or custom layer adjustments.

To overcome this, the most optimal solution was to rebuild the model architecture using Python, trying to be the most accurate and closest to the original one. This approach allowed to regain full control over the model's layers and weights, enabling fine-tuning, architectural adjustments, and integration of transfer learning strategies.

3.5 Development of a reconstructed model

Since the original model was not available in a reusable format, it was reconstructed from scratch. In doing so, the objective was to replicate the original architecture, features, and parameters as closely as possible to those used in the DeVCopy pipeline (Doorn, R. B., 2022). As a result, this reconstruction also inherited certain methodological limitations present in the initial model.

The process began with preprocessing the input data, including the selection of relevant features and the definition of the target variable for training. It selected the most informative features from the metadata tables, as described in the previous work by Doorn (Doorn, R. B., 2022). These features included GC content, RepeatMasked information, depth of coverage (DoC), standard deviation of DoC, and the log R Ratio.

3 Methodology

For the target variable, it was used the variant column, which contains validated CNV annotations for each genomic region. To enable binary classification, duplications (DUP) and deletions (DEL) were grouped under a unified label, "VAR".

To remain consistent with the original model's architecture, the TabNetClassifier from the PyTorch library was used (4.1.0) (Arik & Pfister, 2021). The dataset was split into training and testing subsets following an 80/20 ratio, allocating 80% of the data for training. Then, it also used the Adam optimizer (*Adam — PyTorch 2.4*, n.d.) with a learning rate of 0.01.

Model training included early stopping to prevent overfitting, with a patience of 50 epochs. The maximum number of epochs was set to 100. The batch size was adjusted depending on the dataset: 265 for the Twist datasets and 20 for datasets containing validated copy number variants.

3.5.1 Assessment of the reconstructed model generalizability

To evaluate the generalizability of the reconstructed model, a validation script was developed to perform inference across multiple datasets. First, the model was tested on a dataset from the same batch to establish a baseline performance. Next, it was evaluated on the ICR96 dataset to assess its ability to generalize to targeted panels generated using a different sequencing technology. Finally, the model was tested on a whole-exome sequencing (WES) dataset to determine whether increased data volume could further enhance its performance.

For this was used the reconstructed Tabnet model explained before, that was stored in a zip file. The model was then initialized using TabNetClassifier from the Pytorch TabNet library (4.1.0) (Arik & Pfister, 2021), with the Adam optimizer (*Adam — PyTorch 2.4*, n.d.) and a learning rate of 0.01, consistent with the original training configuration. The input datasets were loaded including the same genomic features as in the training. Class labels were encoded using a previously saved LabelEncoder (label_encoder.pkl), using joblib (*Joblib 1.4.2*, n.d.), to maintain alignment with the model's expected input format.

Predictions were generated for the 20% of the dataset and performance metrics were computed using classification_report from Scikit-learn (*Scikit-Learn 1.3.0*, n.d.). The report

3 Methodology

was saved as a CSV file for downstream analysis, and to visualize classification performance, a confusion matrix was plotted and exported.

3.6 Application of Domain Adaptation Techniques

Once the reconstructed model demonstrated comparable performance to the original and was properly validated on independent datasets, transfer learning techniques were applied to increase its effectiveness. In particular, domain adaptation methods such as Kernel Mean Matching (KMM) and fine-tuning were used.

3.6.1 Kernel Mean Matching (KMM)

This method aims to align the distribution of the training data (source domain) with that of a new target domain, in this case into the ICR96 and Nimblegen (NG) datasets. After loading and preprocessing the new target data in the same format, KMM was used to assign weights to the training samples, allowing the model to learn patterns that are more representative of the target distribution.

The data was then split into training, validation, and test sets. An initial 80/20 train-test split was applied, followed by a second 80/20 split within the training set to create a validation subset. This approach ensured enough samples in each set for robust evaluation and tuning.

For model selection, the TabNetClassifier from the PyTorch library (4.1.0) (Arık & Pfister, 2021) was used to remain consistent with the architecture of the original implementation. The classifier was configured with the Adam optimizer (*Adam — PyTorch* 2.4, n.d.) and a learning rate of 0.01. During training, early stopping was employed with a patience of 10 epochs to prevent overfitting, and the maximum number of training epochs was set to 50. The batch size was fixed at 256, with a virtual batch size of 128 to reduce memory consumption. The previously calculated sample weights from the KMM step were incorporated directly into the training procedure to influence the model's optimization process.

3 Methodology

After training, the model was evaluated on the target domain. Classification metrics such as precision, recall, and F1-score were computed and displayed using a detailed classification report. A confusion matrix was also generated and saved to visualize the distribution of true and predicted labels.

3.6.2 Fine tuning

The goal was to refine the model's performance on samples derived from an unseen distribution, while preserving the previously learned representations. This process leveraged transfer learning through additional supervised training on the new data.

To ensure label consistency with the original model, the previously saved LabelEncoder was reloaded using joblib (*Joblib 1.4.2*, n.d.). This allowed the new target labels to be transformed in the same way as during initial model training, maintaining coherence between old and new data. The feature values were extracted into NumPy arrays, and the dataset was then split into training and validation subsets using an 80/20 ratio.

The reconstructed model is then loaded. For fine-tuning, the same architecture was used, and the model was recompiled with the Adam optimizer, this time with a reduced learning rate of 0.0001. The lower learning rate was chosen deliberately to allow gradual adjustments in the model's weights, preventing drastic changes that could erase previously acquired knowledge.

3.7 Methodological Error Analysis

Following the initial validation of the model, a critical review of its experimental design was conducted to assess the robustness and real-world applicability of the reported results. One error identified through the analysis of the model was that the initial downsampling done in the process of merging the information was not random between regions and samples, as in the final metadata table used for the input all the normal regions were from the first sample.

Nevertheless, as commented before, the use of class balancing strategies, specifically, this downsampling of normal regions during dataset construction is an

4 Results

important methodological limitation by itself. While this approach is beneficial during training to mitigate class imbalance, its application to the test set may have artificially inflated the performance metrics.

To obtain a more realistic understanding of the model’s behavior under real-world conditions, a re-evaluation was carried out using the original class distribution in the test set, without any downsampling. In this updated evaluation, the test data preserved the natural imbalance, where normal regions vastly outnumber true CNV-containing regions. This allowed for a more accurate simulation of clinical diagnostic scenarios, in which CNVs are rare and difficult to detect.

To achieve this, the R script for the calculation of the Log R Ratio in the metadata extraction pipeline was changed. This script undergoes a downsampling approach before the calculation of LRR, the script was modified by simply removing this line from the code, maintaining the rest of the script as the original one.

4 Results

The results of validation of improvement of the model, including quantitative performance metrics and analysis of feature importance, are detailed in the following sections

4.1 Validation of the original model

First, to ensure a proper validation of the model, it was necessary to confirm that it worked as properly as in the original project. For this purpose, it followed the same methodology used in the dataset with the same characteristics: the 202502 batch of the Twist dataset (with artificial class balancing). The validation results are presented in *Table 1*.

Table 1. Performance of the original model on the same datasets used for its development and evaluation in the original publication. Validation was performed on the Twist dataset (Batch 202502), which includes artificial class balancing. The table reports overall accuracy for each dataset, as well as class-specific precision, recall, and F1-score for the “Normal” and “CNV” classes. These metrics reflect the model’s effectiveness in correctly identifying normal samples and those with copy number variations.

4 Results

Datasets	Accuracy	Class	Precision	Recall	F1-score
Batch 202502	0.96	Normal	0.96	0.96	0.96
		CNV	0.96	0.96	0.96

As observed, the model demonstrated excellent performance on the Twist dataset, achieving high accuracy and F1-score (0.96). However, the performance on the ICR96 dataset was substantially lower, particularly in the detection of CNV, where the F1-score dropped to zero (*Table 1*). These results are consistent with the original project, where in the Twist dataset achieved an F1-score of 0.98 (Doorn, R. B., 2022).

After confirming the replication of the original model's performance, additional experiments were conducted using further balanced datasets to investigate whether dataset balancing could enhance generalization to other datasets. The model was first evaluated using another Twist dataset from the same probe design and sequencing technology to assess reproducibility. Then, it was tested on the ICR96 dataset, with a different gene panel and probe design, and finally on the NG dataset, a whole-exome sequencing (WES) dataset generated using the Roche NimbleGen platform for probe design. The results of these evaluations are summarized in *Table 2*.

Table 2. The results show overall accuracy and per-class metrics (Precision, Recall, and F1-score) for three different datasets: Batch 202503, ICR96, and NG. Metrics are reported for two classes: *Normal* and *CNV*. Performance is consistently high for the Batch 202503, moderate for ICR96, and low for NG, with low scores for the *Normal* class and higher scores for *CNV*. These results can indicate potential class imbalance or challenges in generalizing across datasets.

Datasets	Accuracy	Class	Precision	Recall	F1-score
Batch 202503	0.94	Normal	0.94	0.94	0.94
		CNV	0.94	0.94	0.94
ICR96	0.55	Normal	0.53	0.55	0.54
		CNV	0.56	0.55	0.56
NG	0.45	Normal	0.06	0.26	0.09
		CNV	0.84	0.47	0.6

4 Results

The second batch of Twist data again demonstrated strong performance with an overall accuracy of 0.94 and a F1-score also of 0.94 (*Table 2*). Nevertheless, even after applying undersampling to balance the datasets, both ICR96 and NG (WES dataset) continued to exhibit poor results. ICR96, although showing a better performance than NG, has an accuracy of 0.55 and a F1-score of 0.54 for the Normal Class and 0.56 for CNVs (*Table 2*), low metric to provide a decent prediction. For the NG dataset, the performance decrease a lot, and although the accuracy is similar to the one in ICR96 the results shows a particularly low F1-score of 0.09 for the Normal Class (*Table 2*), highlighting the model's difficulty in generalizing beyond the Twist dataset

4.1.1 Analysis of Model Generalizability

Since the model performed robustly on the Twist datasets, further analysis focused on evaluating its generalizability to the ICR96 and NG datasets, where performance declined significantly. Two strategies were tested: first, using DeCoN to validate CNVs instead of relying on MLPA results; and second, filtering the datasets to include only the genomic regions used during the model's initial training. A detailed explanation of this analysis is provided in Section 3.4.2. The results are summarized in *Table 3*.

Table 3. This table presents the model's performance on two datasets (ICR96 and NG) using two different evaluation strategies: (1) in silico validation of CNVs using DeCoN for in silico validation of CNVs, and (2) filtering the datasets to include only genomic regions used during model training. Accuracy and per-class metrics (Precision, Recall, and F1-score) are reported for both the Normal and CNV classes. While DeCoN-based validation has moderate performance on both datasets, applying the region-filtering approach results in lower overall accuracy and a marked decline in the model's performance.

Analysis	Dataset	Accuracy	Class	Precision	Recall	F1-score
DECoN	ICR96	0.56	Normal	0.54	0.57	0.55
			CNV	0.58	0.56	0.57
	NG	0.51	Normal	0.11	0.52	0.18
			CNV	0.9	0.5	0.65
Filtered	ICR97	0.4	Normal	0.15	0.3	0.2
			CNV	0.66	0.43	0.52
	NG	0.49	Normal	0.02	0.04	0
			CNV	0.98	0.5	0.66

4 Results

By using DECoN for CNV validation, the model achieved moderate performance on both ICR96 and NG datasets, with overall accuracies of 0.56 and 0.51, respectively (*Table 3*). On ICR96, both classes had comparable metrics, having a F1-score of 0.55 for the Normal class and 0.57 for CNVs. However, on the NG dataset, performance was unbalanced, with the CNV class achieving high precision (0.90) but only moderate recall (0.50) (*Table 3*), and the Normal class suffering from low precision (0.11) and F1-score (0.18) (*Table 3*), reflecting challenges in model generalization.

To assess whether the model could at least classify correctly the regions it had been trained on, the datasets were filtered to include only those regions. When filtering both datasets to include only genomic regions present in the training data, overall accuracy decreased to 0.40 for ICR96 and 0.49 for NG (*Table 3*). The Normal class performance dropped sharply, with an F1-score of 0.20 on ICR96 and effectively zero on NG (*Table 3*), while the CNV class maintained relatively higher precision but moderate recall and F1-scores. These results suggest that restricting the evaluation to training regions did not improve generalization and disproportionately impaired detection of the Normal class.

Using *in silico* validation with DECoN as the CNV reference led to minimal improvements in model performance, with slight increases in overall metrics but no significant impact on F1-scores or accuracy. The most noticeable change was in the WES dataset, where the Normal class F1-score improved modestly from 0.09 (*Table 2*) to 0.18 (*Table 3*). In contrast, filtering the datasets to include only regions previously seen during training resulted in a worse performance for both ICR96 and WES datasets, with the Normal class F1-score dropping sharply from 0.54 (*Table 2*) to 0.20 (*Table 3*) in ICR96 and from 0.09 (*Table 2*) to directly zero (*Table 3*) in the NG dataset.

4.2 Validation of the reconstructed model

After analysing the initial results, it was decided to implement transfer learning techniques to evaluate whether adapting the model to the target domain could improve performance without the need to retrain the original model. However, access to the original model was unavailable. Therefore, a new model was developed, aiming to replicate the

4 Results

behaviour of the original as closely as possible based on the available information, further information of the development of the reconstructed in Section 3.5 .

Once the new model was created, it was essential to validate that its performance aligned with the original model. To do this, a series of tests were conducted.

Table 4. Training performance of the reconstructed model and the original model with two datasets (Batch 202502 and Batch 202503). It is also used a combination of both batches for observing the effect of data augmentation in the model's performance. Accuracy and per-class metrics (Precision, Recall, and F1-score) are reported for the *Normal* and *CNV* classes. Although the original model shows higher accuracy and more balanced class metrics, the reconstructed model still maintains relatively strong performance across both datasets and classes.

Model	Dataset	Accuracy	Class	Precision	Recall	F1-score
Reconstructed model	Batch 202502	0.9	Normal	0.87	0.94	0.9
			CNV	0.93	0.87	0.9
	Batch 202503	0.88	Normal	0.86	0.92	0.89
			CNV	0.91	0.85	0.88
	Batch 202502 + 202503	0.9	Normal	0.89	0.91	0.89
			CNV	0.91	0.88	0.89
Original model	Batch 202502	0.96	Normal	0.96	0.96	0.96
			CNV	0.96	0.96	0.96
	Batch 202503	0.94	Normal	0.94	0.94	0.94
			CNV	0.94	0.94	0.94

In general, the reconstructed model showed slightly lower metrics compared to the original, with accuracies ranging from 0.88 to 0.90 across the batches (*Table 4*). Precision and recall for the *Normal* class varied between 0.86 and 0.89, while for the *CNV* class, values ranged from 0.85 to 0.93 (*Table 4*). Despite these small decreases, the reconstructed model's performance remains strong, especially when considering other datasets where accuracy was around 0.5 (*Table 2*).

It can be noticed that combining the two training batches did not result in an improvement in model performance. The model trained on the combined batches achieved an accuracy of 0.90, which was the same as the accuracy of the model trained solely on Batch 202502 (*Table 4*). Similarly, the F1-scores were very close, with 0.89 for the combined batch

4 Results

and 0.90 for Batch 202502 alone (*Table 4*). This indicates that increasing the training dataset size by merging batches did not increase predictive performance.

To further inform the choice of the optimal training batch, additional validation tests were conducted on external datasets.

4.2.1 Assessment of Model Generalizability

These models were further tested on ICR96 and NG datasets, both individually trained on separate batches and on a merged dataset, to assess if additional training data could improve performance. Results are shown in *Table 5*.

Table 5. Performance comparison of models trained on individual datasets (Batch 202502 and Batch 202503) and their combination. Accuracy and per-class metrics (Precision, Recall, and F1-score) are reported for the Normal and CNV classes on the ICR96 and NG datasets. The model trained on Batch 202502 showed the best overall performance on ICR96, while the combined model improved prediction of the Normal class in NG. However, combining both batches led to a noticeable decrease in CNV detection performance.

Model	Dataset	Accuracy	Class	Precision	Recall	F1-score
Batch 202502	ICR96	0.64	Normal	0.65	0.59	0.62
			CNV	0.62	0.68	0.65
	NG	0.43	Normal	0.3	0.05	0.08
			CNV	0.48	0.89	0.63
Batch 202503	ICR96	0.6	Normal	0.68	0.39	0.5
			CNV	0.57	0.82	0.67
	NG	0.49	Normal	0.17	0.01	0.01
			CNV	0.49	0.97	0.66
Batch 202502 + 202503	ICR96	0.48	Normal	0.49	0.87	0.63
			CNV	0.42	0.09	0.15
	NG	0.53	Normal	0.52	1	0.68
			CNV	0.99	0.06	0.12

Models trained on individual batches showed differing performance across datasets. The model trained on Batch 202502 achieved the highest overall accuracy on the ICR96 dataset (0.64), with balanced F1-scores of 0.62 for the Normal class and 0.65 for CNVs. However, performance on the NG dataset was considerably weaker, particularly for the Normal class, which had a very low F1-score of 0.08 (*Table 5*).

4 Results

The model trained on Batch 202503 had slightly lower accuracy on ICR96 (0.60), with a more uneven distribution between classes, F1-score of 0.50 for Normal and 0.67 for CNVs. On NG, performance improved slightly with an accuracy of 0.49, but the F1-score for the Normal class dropped further to 0.01 (*Table 5*).

When the training data from both batches (202502 + 202503) was combined, performance on ICR96 decreased with an accuracy of 0.48 but the class performance changed: the Normal class F1-score improved to 0.63, but the CNV class dropped to 0.15. On the NG dataset, this combined model performed best for the Normal class with an F1-score of 0.68, but showed a huge decrease in CNV detection, with an F1-score of only 0.12 (*Table 5*). These results suggest that combining datasets can benefit one class (typically the Normal class) at the cost of reducing performance on the other, particularly for CNVs.

Additional analyses were performed to evaluate the clinical relevance of these results by analyzing false positive and false negative rates across the different datasets.

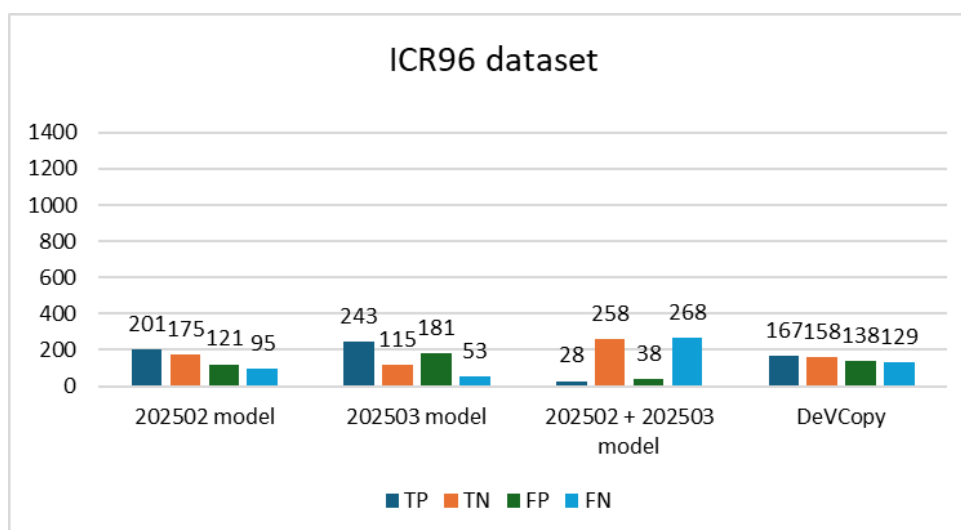


Figure 4. Comparison in metrics between the different training models for the ICR96 dataset . Bar plot display true and false positives, and true and false negatives for the different training models in the ICR96 dataset. While in the model trained with a combination of the batches TN and FN have a notable more relevance, in the rest of the training all metrics seem consistent.

In the case of ICR96, only in the model trained with a combination of both batches the model shows a remarkable increase of false negatives with 268 FN, versus only 38 FP (*Figure 4*). In the rest of the training models although no clear pattern emerged for performance

4 Results

degradation, is it consistent that FP are always higher than FN. For the model trained with the 202502 batch it shows 121 FP vs 95 FN, 181 FP vs 53 FN for the 202503 batch and finally 138 FP vs 129 FN (*Figure 4*) in the original model.

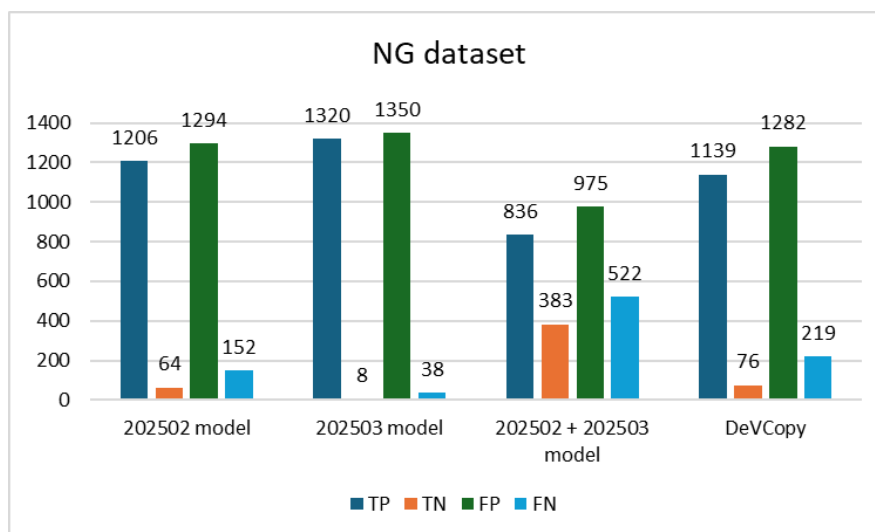


Figure 5. Comparison in metrics between the different training models for the NG dataset. Bar plot display true and false positives, and true and false negatives for the different training models in the NG dataset. For the NG datasets the pattern is clear, there is a notable increase of false positives across all training models, including the original model. The only model that mitigates this imbalance in the one trained with a combination of the two batches.

In contrast, for the WES dataset, a clear dominance of false positives was observed contributing most significantly to the performance drop. In most of the models the false positive number is always above 1200 and true positive always above 1000, while dropping in TN (< 80) and FN (< 250) (*Figure 5*). The only model that mitigates this imbalance is the one trained with a combination of the batches where, although FP (975) and TP (836) are still higher than the rest, TN and FN increase with a number of 383 and 522 respectively (*Figure 5*).

In addition, a further analysis was done to see if the model could improve. This analysis was done by separating the CNV class into two classes: DUP and DEL. This was done to take into account the differences in coverage and LRR in both classes. The results are summarized in the following table:

Table 6. Accuracy and per-class metrics (Precision, Recall, and F1-score) are reported for the Normal, DEL and DUP classes on the ICR96 and NG datasets. Overall, the model showed moderate performance on the

4 Results

ICR96 dataset across all categories, while results on the NG dataset revealed substantial variability between classes, with reduced effectiveness particularly in detecting normal and deletion events.

Datasets	Accuracy	Class	Precision	Recall	F1-score
ICR96	0.61	Normal	0.65	0.6	0.63
		DUP	0.44	0.7	0.54
		DEL	0.69	0.6	0.64
NG	0.29	Normal	0.25	0.05	0.08
		DUP	0.75	0.5	0.6
		DEL	0.07	0.87	0.13

On the ICR96 dataset, the model achieved a balanced performance with an overall accuracy of 0.61, and F1-scores of 0.63, 0.54, and 0.64 for the *Normal*, *DUP*, and *DEL* classes respectively (*Table 6*). In contrast, performance on the NG dataset was significantly lower, with an overall accuracy of 0.29. While the model maintained moderate performance on *DUP* (F1-score: 0.60), it struggled with the *Normal* (0.08) and *DEL* (0.13) classes (*Table 6*), highlighting the challenges of the model in generalizing to data from different sources or distributions.

4.3 Domain Adaptation techniques

To address the lack of generalization, domain adaptation techniques were explored, specifically Instance Weighting and Fine-tuning.

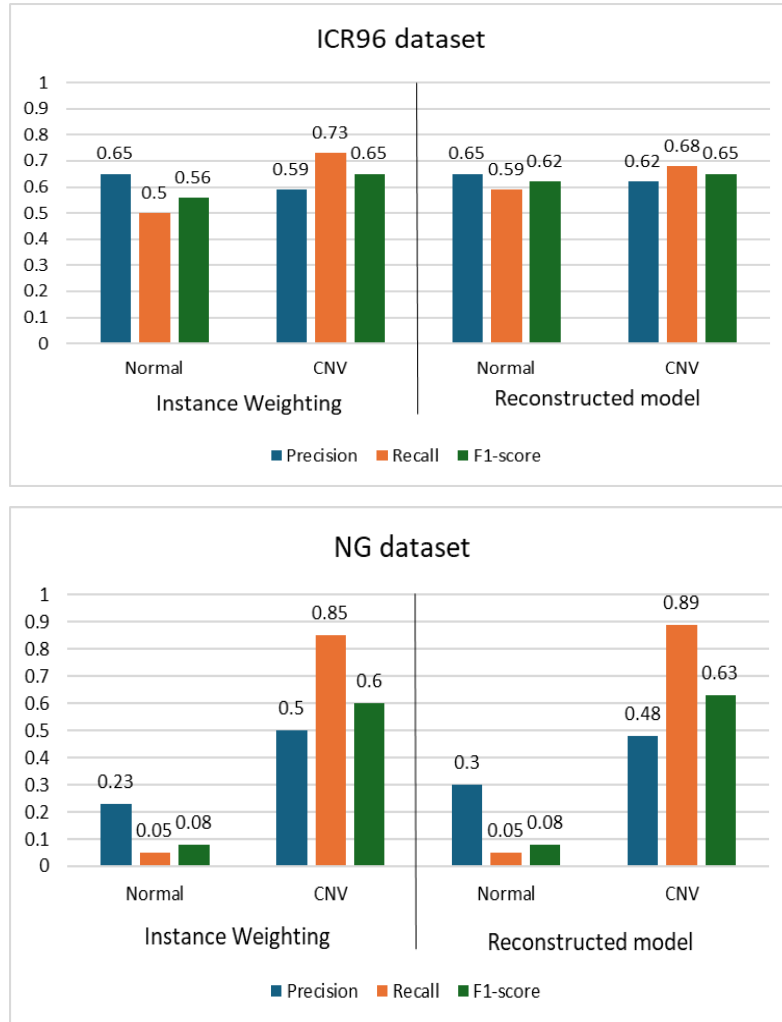


Figure 6. Comparison of the performance of the model before and after Instance Weighting (KMM) in ICR96 and NG .Bar plots display precision, recall, and F1-score for the Normal and CNV classes after applying instance-weighting (left in each subplot) compared to the reconstructed model without adaptation (right in each subplot). In ICR96, instance weighting slightly improved the CNV class performance while maintaining similar metrics for the Normal class. In contrast, for NG, instance weighting resulted in limited improvement for the Normal class, though it preserved relatively strong performance for the CNV class.

For Instance Weighting using Kernel Mean Matching (KMM), the results do not indicate a meaningful improvement in model performance. In the ICR96 dataset, it slightly improved performance for the *CNV* class (F1-score: 0.73 vs. 0.65) and yielded similar results for the *Normal* class (F1-score: 0.59 vs. 0.62). In the NG dataset, instance weighting showed limited benefits for the *Normal* class, with very low recall (0.05) and F1-score (0.08), while the *CNV* class achieved an F1-score of 0.60 (Figure 6). This suggests that incorporating domain

4 Results

distribution through instance weighting does not improve the model's ability to generalize to new data.

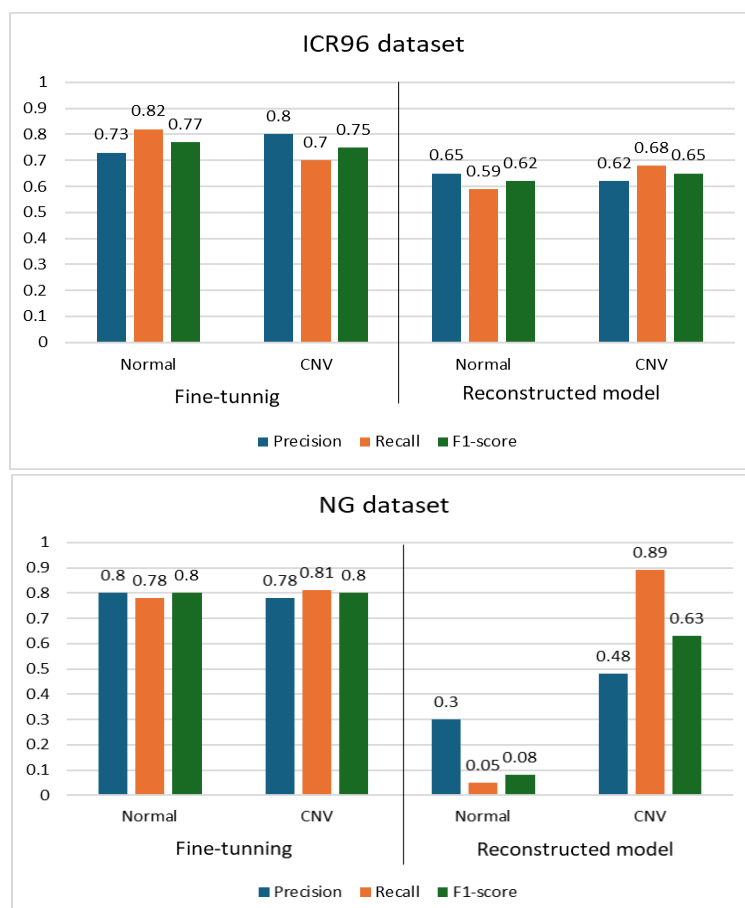


Figure 7. Comparison of the performance of the model before and after Fine-tuning in ICR96 and NG. Bar plots show precision, recall, and F1-score for the *Normal* and *CNV* classes after applying fine-tuning (left in each subplot) compared to the reconstructed model without adaptation (right in each subplot). In the ICR96 dataset, fine-tuning improved performance across both classes, compared to the baseline model. In the NG dataset, fine-tuning substantially increased the performance for both classes, highlighting fine-tuning as an effective domain adaptation strategy.

In contrast, fine-tuning demonstrates a noticeable improvement in performance. For the ICR96 dataset, the most significant gain is observed in the recall of the normal class, which increases by 0.23, from 0.59 to 0.82, resulting in an average F1-score of 0.76 compared to the initial 0.62 (Figure 7).

The improvement is even more pronounced in the WES dataset, where accuracy increases from 0.43 to 0.79 and the average F1-score rises from 0.35 to 0.8. This enhancement is

4 Results

primarily driven by improved detection of the normal class, with precision improving from 0.3 to 0.8, and recall from 0.05 to 0.78 (*Figure 7*).

However, despite these improvements, it is important to note that fine-tuning requires retraining the model on the entire domain dataset, which may limit its practicality for real-world scenarios where large amounts of labeled target data may not be available. To assess the feasibility of fine-tuning with limited data, additional experiments were conducted using only single samples from the target domain.

Table 7. Performance of fine-tuning in a single sample of both datasets. Accuracy and per-class metrics (Precision, Recall, and F1-score) are reported for the Normal and CNV classes on the ICR96 and NG datasets. Single samples have a decrease in accuracy but remain moderate for both samples. Nevertheless, an important decrease in the F1-score of the Normal class, specially in the NG dataset, is shown, showing class imbalance.

Dataset	Accuracy	Class	Precision	Recall	F1-score
ICR96 sample	0.61	Normal	1	0.2	0.37
		CNV	0.56	1	0.7
NG sample	0.5	Normal	0	0	0
		CNV	0.5	1	0.6

While fine-tuning increases the model's overall performance when sufficient data is available, applying it to a single sample limits its benefit, primarily due to the lack of representative data. The most significant decline is observed in the normal class, where the F1-score drops from 0.77 to 0.37 for the ICR96 sample, and from 0.8 to 0 for the WES sample (*Table 7*).

4.4 Feature importance

To gain a deeper understanding of the biological significance in these results, feature importance was calculated across all training datasets. This analysis aims to identify which features contributed most significantly to the model's performance. The results of this analysis are in *Figure 8*.

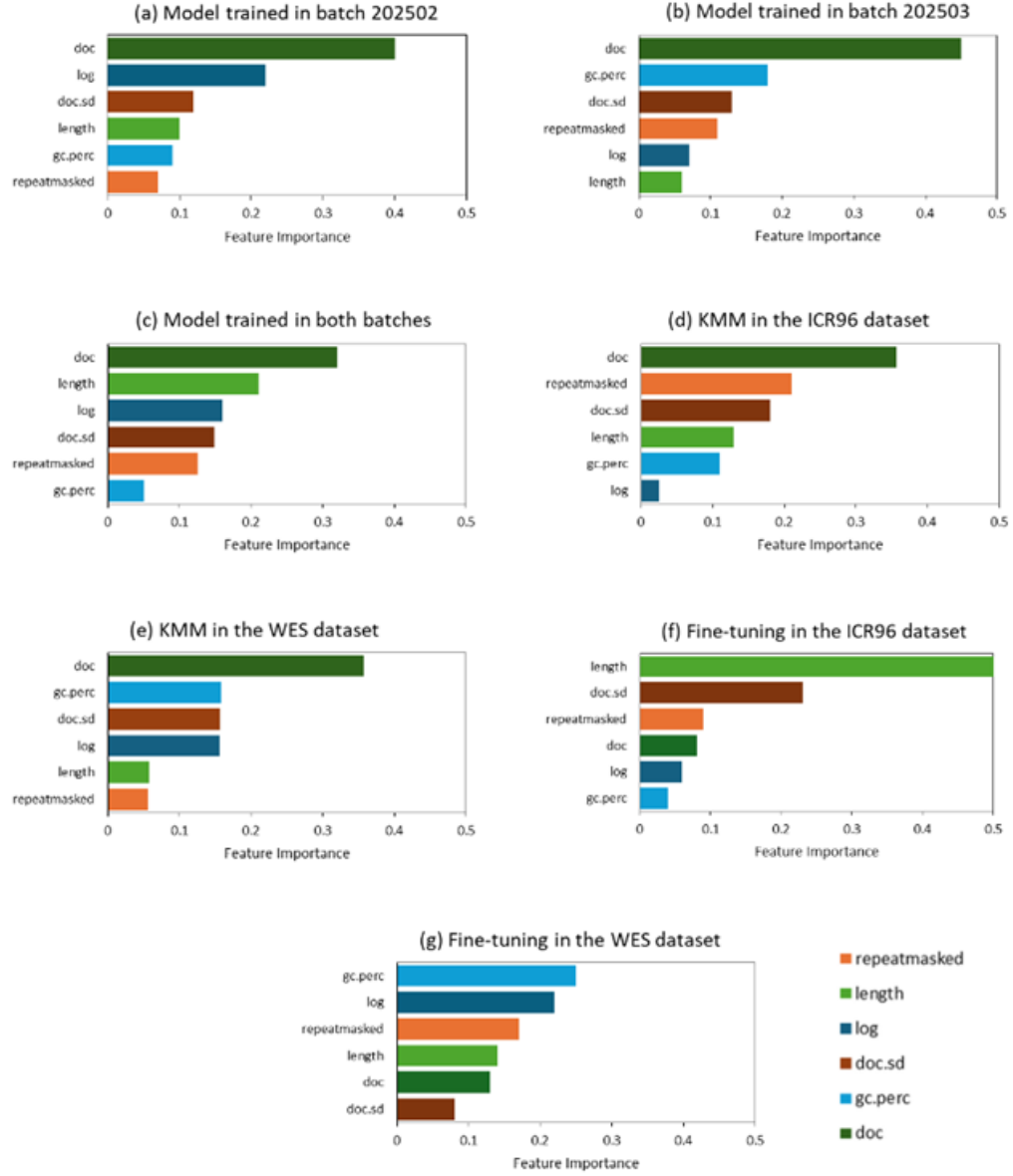


Figure 8. Feature importance in each test. (a), (b) and (c), show training of the model in the Twist dataset with doc being in all cases the most important feature. (d) and (e) shows the feature importance in the retraining with Instance weighting where doc continues showing more relevance to the features. (f) and (g) shows the retraining with fine-tuning where the most important feature for ICR96 is length with difference and gc.perc for the WES dataset.

Analysis of feature importance highlights that doc remains the most influential feature when training on the Twist dataset. However, after fine-tuning on the validation datasets, a substantial shift in feature importance is observed. Specifically, length becomes the most relevant feature for ICR96, while gc.perc increases prominence for the WES dataset. These

4 Results

changes underline the distinct feature distributions across datasets and demonstrate how fine-tuning not only adjusts model weights but also alters feature relevance according to domain-specific characteristics.

4.5 Imbalanced tests

Nevertheless, as commented before all these results are biased due to the under sampling of the normal class. While this approach facilitates model development and evaluation, it does not fully represent real-world clinical scenarios, where the class distribution is heavily skewed towards the *Normal* class. In order to assess the model's true performance under realistic clinical conditions, additional experiments were conducted using imbalanced datasets. Two approaches were evaluated: Training on balanced data and validating on imbalanced data and training directly on unbalanced data.

4.5.1 The reconstructed model validated in imbalanced data

The first test involved utilizing the new model it was created and trained from the batch 2020502 and evaluating its performance on naturally unbalanced datasets.

Table 8. Model performance across three datasets: Twist, ICR96, and NG. Accuracy and per-class metrics (Precision, Recall, and F1-score) are reported for the Normal and CNV classes. While the model performs strongly on the Twist dataset, with high accuracy and balanced metrics for the Normal class, it struggles to detect CNVs, as shown by low precision and F1-scores.

Dataset	Accuracy	Class	Precision	Recall	F1-score
Twist dataset	0.93	Normal	0.99	0.93	0.96
		CNV	0.09	0.84	0.16
ICR96 dataset	0.60	Normal	0.99	0.61	0.76
		CNV	0.003	0.59	0
NG sample	0.99	Normal	0.99	0.99	0.99
		CNV	0	0	0

As observed, while the model maintained excellent performance for the *Normal* class across all datasets, its ability to correctly classify CNVs drastically declined, with F1-scores close to zero on all datasets, even in the Twist dataset where with artificial balancing achieves excellent results (*Table 8*).

5 Discussion

4.5.2 Model Retraining on Imbalanced Data

In the second test, the model was trained directly on the unbalanced datasets without any artificial class balancing.

Table 9. Model performance on the Twist, ICR96, and NG datasets. Accuracy and per-class metrics (Precision, Recall, and F1-score) are reported for the Normal and CNV classes. The model demonstrates near-perfect performance for the Normal class across all datasets but close to 0 metrics for CNVs in all datasets, indicating the strong imbalance of classes.

Dataset	Accuracy	Class	Precision	Recall	F1-score
Twist dataset	0.99	Normal	0.99	0.99	0.99
		CNV	0.78	0.017	0.032
ICR96 dataset	0.99	Normal	0.99	0.999	0.998
		CNV	0	0	0
NG sample	0.99	Normal	0.99	0.99	0.99
		CNV	0	0	0

Training on unbalanced data led to excellent performance for the dominant *Normal* class but completely failed to detect *CNV* events in both ICR96 and NG samples. Even in the Twist dataset, where the model was originally trained, the CNV recall remained extremely low (0.017) (*Table 9*), with a severe bias towards the majority class when trained on imbalanced data.

5 Discussion

The detection of CNVs using machine learning models presents a complex challenge, particularly when going from controlled synthetic environments to real-world clinical datasets. This study examines the performance, limitations, and potential of the CNV detection model developed by R.Doorn (hereafter referred to as the original model), across various datasets, alongside the role of domain adaptation, dataset balancing, and feature importance in influencing model behavior.

The initial step in validating the project methodology was to replicate the results of the original CNV detection model using datasets with the same characteristics as those used to

5 Discussion

train the original model. As shown in *Table 1*, the original model achieved a near-perfect performance on the Twist dataset (artificially balanced), with an accuracy and F1-score of 0.96 across both Normal and CNV classes. This reaffirms the replicability of the original model in the same environment.

When the model was tested on two external datasets, ICR96 (targeted sequencing) and NG (whole-exome sequencing), performance decreased. In the ICR96 dataset, F1-score fell to 0.54 for the normal class and 0.56 for CNVs (*Table 2*). The situation was more pronounced in the NG dataset, where the normal class F1-score dropped to 0.09 (*Table 2*), highlighting a marked imbalance in classification accuracy. This suggests that the model may not be capturing fundamental biological markers, but instead learning domain-specific patterns unique to the Twist datasets. Similar domain-specific overfitting behaviors have been observed in previous studies (Shao *et al.*, 2019).

These results illustrate a classic manifestation of domain shift, a common limitation in machine learning where models trained on data from one domain, in this case a specific sequencing platform, fail to generalize to another. This is especially critical in clinical genomics, where pipelines must work across diverse sequencing technologies and experimental designs (Guan & Liu, 2022).

To further explore this issue, two other strategies were implemented: the use of an alternative CNV caller (DECoN), and region-based filtering. In this context, DECoN was used as the reference tool to define the true CNVs instead of relying on MLPA-based validation, since DECoN was also the tool originally used to label the training data. The DECoN-based approach slightly improved performance, with the ICR96 dataset reaching an F1-score of 0.57 (*Table 3*) and the NG dataset improving from 0.09 (*Table 2*) to 0.18 (*Table 3*) in the normal class. The slight performance gain observed with DECoN can be attributed to its tendency to call a larger number of CNVs than those clinically validated. As a result, when undersampling is applied to balance the classes, the CNV class includes a broader range of CNV-like features, introducing more variability and representation in the training data. This increased variability can help the model generalize slightly better, though it does not reflect improved clinical accuracy as many of these additional CNVs may be false positives.

5 Discussion

Region-based filtering, aimed at restricting the analysis to genomic regions where the model was initially trained. Nevertheless, it actually worsened its performance, dropping the F1-score of the normal class to zero in the NG dataset (*Table 3*). These results highlight that superficial adaptation techniques, such as changing the CNV caller or limiting the analysis to specific genomic regions, fail to address the underlying representational and distributional differences between domains (Kouw & Loog, 2021).

Given the limitations in the accessibility of the original model, a reconstructed version was developed and evaluated on datasets with the same characteristics as those used to train the original models. Performance on the Twist datasets remained strong, although slightly reduced (F1-score = 0.9) (*Table 4*), verifying functional replication. However, the reconstructed model equally replicated the same generalization shortcomings on all datasets as F1-scores remained low across both classes in ICR96 and NG.

Training with a combination of Twist batches (2020502 and 2020503) also failed to produce better generalization. The batches were combined to increase the number of available samples and expand the set of clinically validated regions, with the goal of enhancing model robustness. However, combining them resulted in decreased CNV recall and precision (*Table 5*). Although both batches were generated using the same sequencing protocol and probe design, the observed performance drop may reflect subtle batch-specific patterns or technical noise that the model overfits to. This suggests that even minor heterogeneity within seemingly similar data can harm performance, as discussed by Bengio *et al.* (2013), who noted that adding heterogeneous data without proper alignment can sometimes degrade model generalization.

Interestingly, the decline in generalization performance cannot be totally attributed to a lack of model capacity, because deep learning architectures are well-suited for learning complex patterns (Lecun *et al.*, 2015). Instead, the issue appears to be from overfitting to domain-specific features, such as depth of coverage and GC-content bias, which are known to vary substantially across different sequencing datasets (Benjamini & Speed, 2012). These features, while informative in the source domain, may lose their discriminative power or even become misleading when applied to target domains with different feature distributions.

Error analysis was conducted to evaluate the performance of CNV predictions across different datasets, with the results presented in Section 4.2.1. The analysis revealed that the

5 Discussion

ICR96 dataset showed a mix of false positives and false negatives, suggesting that the model struggled to correctly classify both types of CNVs. In contrast, the NG dataset showed a predominance of false positives for CNVs. High false positive rates are particularly problematic in exome sequencing data, where low and uneven coverage complicates accurate CNV detection (Zare *et al.*, 2017). These error patterns are consistent with recent studies showing that machine learning models trained on synthetically balanced genomic data often fail to generalize well to real-world distributions due to differences in noise and sequencing biases (Spurr *et al.*, 2020).

Although the original model combined both CNV types, duplications (DUP) and deletions (DEL), and performed well on the Twist dataset, its performance was further evaluated on the ICR96 and NG datasets by analyzing these CNV classes separately. This analysis was motivated by the inherent differences in depth of coverage between DUP and DEL regions, as shown in *Figure 1*, where deletions are associated with reduced coverage and duplications with increased coverage. While these distinctions did not affect performance in the source domain, the goal was to assess whether explicitly separating these biologically distinct CNV types could improve predictive accuracy in the target domains. However, as presented in *Table 6*, this approach did not lead to a significant improvement. These findings may also indicate that, despite artificial class balancing, the model may not have been exposed to a sufficient number of DUP and DEL regions during training to effectively learn their distinct patterns and make reliable predictions.

Two domain adaptation strategies were evaluated in order to improve the domain shift of the model: instance reweighting using Kernel Mean Matching (KMM) and supervised fine-tuning on target data. KMM reweights source samples to better match the target domain distribution and, in theory, is the ebay option for domain adaptation in this project as it does not require labeled target data, eliminating the need for a reference batch of labeled samples. However, in this study, KMM proved ineffective (*Figure 6*). This suggests that the feature distributions between domains, particularly between NG and Twist, are too dissimilar for effective alignment using this method.

In contrast, fine-tuning substantially improved performance. On the ICR96 dataset, the F1-score for the Normal class increased from 0.62 to 0.76, while the NG class showed a

5 Discussion

dramatic improvement from 0.35 to 0.80 (*Figure 7*). These results align with findings by Howard & Ruder (2018), who demonstrated that supervised fine-tuning on even a small amount of in-domain data can significantly enhance transfer learning performance.

However, fine-tuning relies on access to labeled target data, which is typically scarce in genomics, and especially so for copy number variants (CNVs) as they are rare events. Even though using 80% of the dataset for training (as explained in Section 3.6.2) led to performance gains, the resulting metrics still doesn't surpass clinical standards as none of the evaluated models surpassed 0.9 in either accuracy or F1-score. This underscores the difficulty of accurately detecting CNVs without pre-existing labeled data.

Attempts to fine-tune using only a single sample further confirmed this limitation. In that case, the F1-score for the Normal class decreased sharply from 0.77 to 0.37 (*Table 7*), indicating that fine-tuning requires a minimum data threshold to be effective. This significantly limits its practicality in real-world clinical settings. As outlined in Section 1, laboratory validation of CNVs is expensive and time-consuming, factors that are incompatible with the speed and efficiency required in clinical environments, where analysis often focuses on a single patient sample or a small batch.

In general, high-quality CNV datasets with validated annotations are extremely limited. This scarcity explains the frequent use of the ICR96 dataset in CNV detection research (e.g., Munté *et al.*, 2024; Barcelona-Cabeza *et al.*, 2021). Although efforts were made to obtain additional datasets from EGA for this study, access was not granted in time to allow for further analysis.

Feature importance analysis across domains provided valuable insights into model behavior. In the Twist datasets, depth of coverage (*doc*) emerged as the most predictive feature (*Figure 8*), which is expected given that *doc* is the primary signal used in conventional CNV detection methods (Li *et al.*, 2024). However, in the ICR96 and NG datasets, fine-tuned models assigned higher importance to other features, such as event length and GC content (*gc.perc*) (*Figure 8*).

This shift may indicate a limitation in the training process: specifically, the lack of explicit modeling of inter-sample variability. When *doc* values are normalized across all

6 Conclusions

samples without considering individual differences, crucial patterns associated with CNV events may be lost. As a result, the model may rely on secondary features that appear more stable across the dataset, even if they are biologically less informative.

These findings suggest that although *doc* should be the dominant feature for CNV detection, improper handling of variability between samples can reduce its apparent importance. Moreover, this connects to a broader issue in CNV analysis, many methods calculate expected read depth using all samples and genomic regions as a reference, rather than tailoring the estimation to each sample individually. This dependency introduces biases and limits generalizability, especially in clinical scenarios where only one sample may be available for analysis.

The final stage of model validation focused on evaluating performance under clinically realistic class distributions. Two approaches were tested: (1) training on balanced data and testing on unbalanced datasets, and (2) training directly on unbalanced data.

In the first scenario, while the model maintained excellent metrics for the Normal class (F1-score > 0.9), its ability to detect CNVs collapsed entirely (*Table 8*). The CNVs F1-scores were close to zero across all datasets, including the Twist dataset. In the NG sample, the CNV class was not detected at all (*Table 8*). This underscores the risk of evaluating models just on balanced datasets, performance may appear strong while being clinically unusable.

Training directly on unbalanced data produced similar results. Although the Normal class achieved nearly perfect metrics, the CNV class again showed zero recall in ICR96 and NG. Even in Twist, where the model should be most familiar, CNV detection remained minimal (recall = 0.017) (*Table 9*). This indicates that the model simply learns to predict the majority class in the absence of balancing, confirming the necessity of stratified learning strategies.

These results have serious clinical implications. In CNV detection, the minority class (CNV) is clinically significant. Any failure to detect CNVs can lead to missed diagnoses of critical genetic conditions.

6 Conclusions

6 Conclusions

This study comprehensively evaluated the performance, generalization capacity, and limitations of machine learning models designed for copy number variation (CNV) detection across multiple genomic datasets. The main conclusions are the following:

1. The original CNV detection model developed by Doorn (2022) was successfully replicated, demonstrating high performance on datasets similar to the training data (Twist), with an F1-score of up to 0.96 under artificially balanced conditions.
2. The model exhibits limited generalization capability when applied to datasets generated using different sequencing strategies and probe designs, highlighting significant domain-specific dependencies.
3. Fine-tuning the model with target domain data improves performance; however, its not good enough for clinical applications and its effectiveness is limited by the insufficient labelled data.
4. The important differences in feature importance across datasets emphasize the critical need for a robust method for difference coverage patterns depending on the probe design.
5. Class imbalance and variability in coverage between sequencing technologies represent the main problem for machine learning approaches in CNV detection.

6.1 Future Directions

Building on the findings of this research, the following avenues are recommended for future work:

- 1 **Development of Robust Domain-Invariant Features:** Future studies could focus on identifying or engineering genomic features that are invariant or less sensitive to sequencing platform biases and batch effects, in order to improve model transferability without extensive fine-tuning.
- 2 **Semi-Supervised and Unsupervised Domain Adaptation:** Given the limitation of labeled CNV data in new domains, exploring semi-supervised or unsupervised domain adaptation techniques (UDA) may enable leveraging unlabeled or weakly labeled data for improved generalization.

7 Bibliography

- 3 **Inter-sample Variability Metadata:** Incorporating normalized coverage features, such as coverage relative to the sample mean, z-score normalization, and log-ratio transformations, could help the model to consider inter-sample coverage variability, potentially improving its robustness and performance in CNV detection.
- 4 **Advanced Class Imbalance Handling Methods:** Novel approaches to handle extreme class imbalance should be evaluated to improve CNV detection sensitivity in real-world clinical settings.
- 5 **Explainable AI for Model Interpretability:** Further exploration of explainability methods such as SHAP (Shapley Additive Explanations) could be used to ensure that model decisions are biologically meaningful, especially when models are fine-tuned or adapted to new domains.
- 6 **Rigorous Clinical Benchmarking:** The clinical implementation of CNV detection tools must be accompanied by rigorous benchmarking against gold-standard validations such as MLPA or aCGH

For further information of the code for development and validation of the reconstructed model and application of domain adaptation techniques visit the github: <https://github.com/beasanesp>.

7 Bibliography

Adam — *PyTorch 2.7 documentation*. (n.d.). Retrieved June 30, 2025, from <https://docs.pytorch.org/docs/stable/generated/torch.optim.Adam.html>

Adolfsson, E., Jonasson, J., Kashyap, A., Nordensköld, A., & Greén, A. (2023). CNV-Z; a new tool for detecting copy number variation in next generation sequencing data. *SoftwareX*, 24, 101530. <https://doi.org/10.1016/J.SOFTX.2023.101530>

7 Bibliography

- Agiannitopoulos, K. (2023). CNV and RNA analysis reveal a germline pathogenic duplication of MSH2 exon 15 in a family with Lynch syndrome: A case report. *Journal of Clinical Images and Medical Case Reports*, 4(2). <https://doi.org/10.52768/2766-7820/2289>
- ALCANTARA, D. D. F. Á., JÚNIOR, S. F. L., ASSUMPÇÃO, P. P. DE, LAMARÃO, L. M., SANT'ANNA, C. D. C., MOREIRA-NUNES, C. A., & BURBANO, R. R. (2022). Identification of Germline Mutations in Genes Involved in Classic FAP in Patients from Northern Brazil. *Cancer Diagnosis & Prognosis*, 2(3), 405. <https://doi.org/10.21873/CDP.10123>
- Arik, S., & Pfister, T. (2021). TabNet: Attentive Interpretable Tabular Learning. *35th AAAI Conference on Artificial Intelligence, AAAI 2021*, 84, 6679–6687. <https://doi.org/10.1609/aaai.v35i8.16826>
- Badillo, S., Banfai, B., Birzele, F., Davydov, I. I., Hutchinson, L., Kam-Thong, T., Siebourg-Polster, J., Steiert, B., & Zhang, J. D. (2020). An Introduction to Machine Learning. *Clinical Pharmacology and Therapeutics*, 107(4), 871. <https://doi.org/10.1002/CPT.1796>
- Barcelona-Cabeza, R., Sanseverino, W., & Aiese Cigliano, R. (2021). isoCNV: in silico optimization of copy number variant detection from targeted or exome sequencing data. *BMC Bioinformatics*, 22(1). <https://doi.org/10.1186/S12859-021-04452-6>,
- bedtools: a powerful toolset for genome arithmetic — bedtools 2.31.0 documentation*. (n.d.). Retrieved June 30, 2025, from <https://bedtools.readthedocs.io/en/latest/>
- Bengio, Y., Courville, A., & Vincent, P. (2013). Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8), 1798–1828. <https://doi.org/10.1109/TPAMI.2013.50>
- Benjamini, Y., & Speed, T. P. (2012). Summarizing and correcting the GC content bias in high-throughput sequencing. *Nucleic Acids Research*, 40(10). <https://doi.org/10.1093/NAR/GKS001>,
- Burdick, K. J., Cogan, J. D., Rives, L. C., Robertson, A. K., Koziura, M. E., Brokamp, E., Duncan, L., Hannig, V., Pfotenhauer, J., Vanzo, R., Paul, M. S., Bican, A., Morgan, T.,

7 Bibliography

- Duis, J., Newman, J. H., Hamid, R., & Phillips, J. A. (2020). Limitations of exome sequencing in detecting rare and undiagnosed diseases. *American Journal of Medical Genetics, Part A*, 182(6), 1400–1406. <https://doi.org/10.1002/AJMG.A.61558>,
- Cheung, E. Y. W., Wu, R. W. K., Li, A. S. M., & Chu, E. S. M. (2023). AI Deployment on GBM Diagnosis: A Novel Approach to Analyze Histopathological Images Using Image Feature-Based Analysis. *Cancers*, 15(20), 5063. <https://doi.org/10.3390/CANCERS15205063/S1>
- Ching, T., Himmelstein, D. S., Beaulieu-Jones, B. K., Kalinin, A. A., Do, B. T., Way, G. P., Ferrero, E., Agapow, P. M., Zietz, M., Hoffman, M. M., Xie, W., Rosen, G. L., Lengerich, B. J., Israeli, J., Lanchantin, J., Woloszynek, S., Carpenter, A. E., Shrikumar, A., Xu, J., ... Greene, C. S. (2018). Opportunities and obstacles for deep learning in biology and medicine. *Journal of the Royal Society Interface*, 15(141). <https://doi.org/10.1098/RSIF.2017.0387>,
- Coutelier, M., Holtgrewe, M., Jäger, M., Flöttman, R., Mensah, M. A., Spielmann, M., Krawitz, P., Horn, D., Beule, D., & Mundlos, S. (2021). Combining callers improves the detection of copy number variants from whole-genome sequencing. *European Journal of Human Genetics* 2021 30:2, 30(2), 178–186. <https://doi.org/10.1038/s41431-021-00983-x>
- Curtis, C., Shah, S. P., Chin, S. F., Turashvili, G., Rueda, O. M., Dunning, M. J., Speed, D., Lynch, A. G., Samarajiwa, S., Yuan, Y., Gräf, S., Ha, G., Haffari, G., Bashashati, A., Russell, R., McKinney, S., Aparicio, S., Brenton, J. D., Ellis, I., ... Caldas, C. (2012). The genomic and transcriptomic architecture of 2,000 breast tumours reveals novel subgroups. *Nature* 2012 486:7403, 486(7403), 346–352. <https://doi.org/10.1038/nature10983>
- de Koning, A. P. J., Gu, W., Castoe, T. A., Batzer, M. A., & Pollock, D. D. (2011). Repetitive Elements May Comprise Over Two-Thirds of the Human Genome. *PLOS Genetics*, 7(12), e1002384. <https://doi.org/10.1371/JOURNAL.PGEN.1002384>
- de Mathelin, A., Atiq, M., Richard, G., de la Concha, A., Yachouti, M., Deheeger, F., Mougeot, M., & Vayatis, N. (2023). *ADAPT: Awesome Domain Adaptation Python Toolbox*. <http://arxiv.org/abs/2107.03049>

7 Bibliography

- Edwin Aco Tito, A., Orlando Hanco Condori, B., & Pérez Vera, Y. (n.d.). *Análisis comparativo de Técnicas de Machine Learning para la predicción de casos de deserción universitaria*. <https://doi.org/10.17013/risti.51.84-98>
- Fowler, A., Mahamdallie, S., Ruark, E., Seal, S., Ramsay, E., Clarke, M., Uddin, I., Wylie, H., Strydom, A., Lunter, G., & Rahman, N. (2016). Accurate clinical detection of exon copy number variants in a targeted NGS panel using DECoN. *Wellcome Open Research*, 1. <https://doi.org/10.12688/WELLCOMEOPENRES.10069.1/DOI>
- Francisco De La Vega, A. M., Irvine, S. A., Anur, P., Potts, K., Kraft, L., Torres, R., Kang, P., Truong, S., Lee, Y., Han, S., Onuchic, V., Han, J., & De La Vega, F. M. (2024). Benchmarking of Germline Copy Number Variant Callers from Whole Genome Sequencing Data for Clinical Applications. *MedRxiv*, 2024.07.12.24310338. <https://doi.org/10.1101/2024.07.12.24310338>
- Gordeeva, V., Sharova, E., Babalyan, K., Sultanov, R., Govorun, V. M., & Arapidi, G. (2021). Benchmarking germline CNV calling tools from exome sequencing data. *Scientific Reports* 2021 11:1, 11(1), 1–11. <https://doi.org/10.1038/s41598-021-93878-2>
- Gowri1, S. G., Devi, R., Sethuraman, K., & Phil, M. (2019). MACHINE LEARNING. *International Journal of Research and Analytical Reviews*. www.ijrar.org
- Guan, H., & Liu, M. (2022). Domain Adaptation for Medical Image Analysis: A Survey. *IEEE Transactions on Biomedical Engineering*, 69(3), 1173–1185. <https://doi.org/10.1109/TBME.2021.3117407>,
- Guffanti, G., Torri, F., Rasmussen, J., Clark, A. P., Lakatos, A., Turner, J. A., Fallon, J. H., Saykin, A. J., Weiner, M., Vawter, M. P., Knowles, J. A., Potkin, S. G., & Macciardi, F. (2013). Increased CNV-Region deletions in mild cognitive impairment (MCI) and Alzheimer's disease (AD) subjects in the ADNI sample. *Genomics*, 102(2), 112–122. <https://doi.org/10.1016/J.YGENO.2013.04.004>
- Guha, T., & Malkin, D. (2017). Inherited TP53 Mutations and the Li–Fraumeni Syndrome. *Cold Spring Harbor Perspectives in Medicine*, 7(4), a026187. <https://doi.org/10.1101/CSHPERSPECT.A026187>

7 Bibliography

- Hakkaart, C., Pearson, J. F., Marquart, L., Dennis, J., Wiggins, G. A. R., Barnes, D. R., Robinson, B. A., Mace, P. D., Aittomäki, K., Andrulis, I. L., Arun, B. K., Azzollini, J., Balmaña, J., Barkardottir, R. B., Belhadj, S., Berger, L., Blok, M. J., Boonen, S. E., Borde, J., ... Walker, L. C. (2022). Copy number variants as modifiers of breast cancer risk for BRCA1/BRCA2 pathogenic variant carriers. *Communications Biology* 2022 5:1, 5(1), 1–15. <https://doi.org/10.1038/s42003-022-03978-6>
- Hill, T., & Unckless, R. L. (2019). A Deep Learning Approach for Detecting Copy Number Variation in Next-Generation Sequencing Data. *G3: Genes|Genomes|Genetics*, 9(11), 3575. <https://doi.org/10.1534/G3.119.400596>
- Hiroi, N., Takahashi, T., Hishimoto, A., Izumi, T., Boku, S., & Hiramoto, T. (2013). Copy number variation at 22q11.2: from rare variants to common mechanisms of developmental neuropsychiatric disorders. *Molecular Psychiatry*, 18(11), 10.1038/mp.2013.92. <https://doi.org/10.1038/MP.2013.92>
- Howard, J., & Ruder, S. (2018). Universal Language Model Fine-tuning for Text Classification. *ACL 2018 - 56th Annual Meeting of the Association for Computational Linguistics, Proceedings of the Conference (Long Papers), 1*, 328–339. <https://doi.org/10.18653/V1/P18-1031>
- Human [Homo sapiens] Genomic Dataset.* (n.d.). Retrieved June 30, 2025, from <https://www.repeatmasker.org/species/hg.html>
- Jiang, Y., Oldridge, D. A., Diskin, S. J., & Zhang, N. R. (2015). CODEX: A normalization and copy number variation detection method for whole exome sequencing. *Nucleic Acids Research*, 43(6), e39. <https://doi.org/10.1093/NAR/GKU1363>,
- Joblib: running Python functions as pipeline jobs — joblib 1.5.1 documentation.* (n.d.). Retrieved June 30, 2025, from <https://joblib.readthedocs.io/en/stable/>
- Knaus, B. J., & Grunwald, N. J. (2023). Manipulate and Visualize VCF Data [R package vcfR version 1.15.0]. *CRAN: Contributed Packages*. <https://doi.org/10.32614/CRAN.PACKAGE.VCFR>

7 Bibliography

- Kouw, W. M., & Loog, M. (2021). A Review of Domain Adaptation without Target Labels. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(3), 766–785. <https://doi.org/10.1109/TPAMI.2019.2945942>
- Lecun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444. <https://doi.org/10.1038/NATURE14539;SUBJMETA=117,639,705;KWRD=COMPUTER+SCIENCE,MATHEMATICS+AND+COMPUTING>
- Li, Y., Yang, F., & Xie, K. (2024). CNV_MCD: Detection of copy number variations based on minimum covariance determinant using next-generation sequencing data. *Digital Signal Processing*, 154, 104680. <https://doi.org/10.1016/J.DSP.2024.104680>
- Liu, Q., Karolak, J. A., Grochowski, C. M., Wilson, T. A., Rosenfeld, J. A., Bacino, C. A., Lalani, S. R., Patel, A., Breman, A., Smith, J. L., Cheung, S. W., Lupski, J. R., Bi, W., & Stankiewicz, P. (2020). Parental somatic mosaicism for CNV deletions – A need for more sensitive and precise detection methods in clinical diagnostics settings. *Genomics*, 112(5), 2937–2941. <https://doi.org/10.1016/J.YGENO.2020.05.003>
- Mahamdallie, S., Ruark, E., Yost, S., Ramsay, E., Uddin, I., Wylie, H., Elliott, A., Strydom, A., Renwick, A., Seal, S., & Rahman, N. (2017). The ICR96 exon CNV validation series: a resource for orthogonal assessment of exon CNV calling in NGS data. *Wellcome Open Research*, 2, 35–35. <https://doi.org/10.12688/WELLCOMEOPENRES.11689.1>
- Malhotra, D., & Sebat, J. (2012). CNVs: harbingers of a rare variant revolution in psychiatric genetics. *Cell*, 148(6), 1223–1241. <https://doi.org/10.1016/J.CELL.2012.02.039>
- Meienberg, J., Bruggmann, R., Oexle, K., & Matyas, G. (2016). Clinical sequencing: is WGS the better WES? *Human Genetics*, 135(3), 359. <https://doi.org/10.1007/S00439-015-1631-9>
- Munté, E., Roca, C., Del Valle, J., Feliubadaló, L., Pineda, M., Gel, B., Castellanos, E., Rivera, B., Cordero, D., Moreno, V., Lázaro, C., & Moreno-Cabrera, J. M. (2024). Detection of germline CNVs from gene panel data: benchmarking the state of the art. *Briefings in Bioinformatics*, 26(1), 645. <https://doi.org/10.1093/BIB/BBAE645>

7 Bibliography

- Neveu, B., Therrien-Laperrière, S., Hay, V., & Amyot, J. (2024). P627: Pool of normal optimization for NGS-based CNV calling in a clinical setting. *Genetics in Medicine Open*, 2, 101533. <https://doi.org/10.1016/J.GIMO.2024.101533>
- Nguyen, L. H., & Holmes, S. (2019). Ten quick tips for effective dimensionality reduction. *PLoS Computational Biology*, 15(6), e1006907. <https://doi.org/10.1371/JOURNAL.PCBI.1006907>
- Pantanowitz, L., Pearce, T., Abukhiran, I., Hanna, M., Wheeler, S., Soong, T. R., Tafti, A. P., Pantanowitz, J., Lu, M. Y., Mahmood, F., Gu, Q., & Rashidi, H. H. (2025). Nongenerative Artificial Intelligence in Medicine: Advancements and Applications in Supervised and Unsupervised Machine Learning. *Modern Pathology*, 38(3), 100680. <https://doi.org/10.1016/J.MODPAT.2024.100680>
- Pei, X. M., Yeung, M. H. Y., Wong, A. N. N., Tsang, H. F., Yu, A. C. S., Yim, A. K. Y., & Wong, S. C. C. (2023). Targeted Sequencing Approach and Its Clinical Applications for the Molecular Diagnosis of Human Diseases. *Cells*, 12(3), 493. <https://doi.org/10.3390/CELLS12030493>
- Phillips, M., Babu, J. R., Wang, X., & Geetha, T. (2020). DNA copy number and structural variation (CNV) contributions to adult and childhood obesity. *Biochemical Society Transactions*, 48(4), 1819–1828. <https://doi.org/10.1042/BST20200556>
- Pinto, D., Delaby, E., Merico, D., Barbosa, M., Merikangas, A., Klei, L., Thiruvahindrapuram, B., Xu, X., Ziman, R., Wang, Z., Vorstman, J. A. S., Thompson, A., Regan, R., Pilorge, M., Pellecchia, G., Pagnamenta, A. T., Oliveira, B., Marshall, C. R., Magalhaes, T. R., ... Scherer, S. W. (2014). Convergence of genes and cellular pathways dysregulated in autism spectrum disorders. *American Journal of Human Genetics*, 94(5), 677–694. <https://doi.org/10.1016/J.AJHG.2014.03.018>
- Pirooznia, M., Goes, F., & Zandi, P. P. (2015). Whole-genome CNV analysis: advances in computational approaches. *Frontiers in Genetics*, 6(MAR), 138. <https://doi.org/10.3389/FGENE.2015.00138>

7 Bibliography

- Plagnol, V., Curtis, J., Epstein, M., Mok, K. Y., Stebbings, E., Grigoriadou, S., Wood, N. W., Hambleton, S., Burns, S. O., Thrasher, A. J., Kumararatne, D., Doffinger, R., & Nejentsev, S. (2012). A robust model for read count data in exome sequencing experiments and implications for copy number variant calling. *Bioinformatics*, 28(21), 2747–2754. <https://doi.org/10.1093/BIOINFORMATICS/BTS526>,
- Posar, A., & Visconti, P. (2020). Neuro-Behavioral Phenotype in 16p11.2 Duplication: A Case Series. *Children*, 7(10), 190. <https://doi.org/10.3390/CHILDREN7100190>
- Pounraja, V. K., Jayakar, G., Jensen, M., Kelkar, N., & Girirajan, S. (2019). A machine-learning approach for accurate detection of copy number variants from exome sequencing. *Genome Research*, 29(7), 1134–1143. <https://doi.org/10.1101/GR.245928.118/-/DC1>
- Povysil, G., Tzika, A., Vogt, J., Haunschmid, V., Messiaen, L., Zschocke, J., Klambauer, G., Hochreiter, S., & Wimmer, K. (2017). panelcn.MOPS: Copy-number detection in targeted NGS panel data for clinical diagnostics. *Human Mutation*, 38(7), 889. <https://doi.org/10.1002/HUMU.23237>
- Ronemus, M., Bradford, D., Laster, Z., & Li, S. (2025). Exploring genome-transcriptome correlations in cancer. *Biochemical Society Transactions*, 53(1), 113–120. <https://doi.org/10.1042/BST20240108>
- Sambamba: process your BAM data faster!* (n.d.). Retrieved June 30, 2025, from <https://lommeriter.github.io/sambamba/index.html>
- Sanders, S. J., Murtha, M. T., Gupta, A. R., Murdoch, J. D., Raubeson, M. J., Willsey, A. J., Ercan-Sencicek, A. G., Di Lullo, N. M., Parikshak, N. N., Stein, J. L., Walker, M. F., Ober, G. T., Teran, N. A., Song, Y., El-Fishawy, P., Murtha, R. C., Choi, M., Overton, J. D., Bjornson, R. D., ... State, M. W. (2012). De novo mutations revealed by whole-exome sequencing are strongly associated with autism. *Nature* 2012 485:7397, 485(7397), 237–241. <https://doi.org/10.1038/nature10945>
- Schwarze, K., Buchanan, J., Fermont, J. M., Dreau, H., Tilley, M. W., Taylor, J. M., Antoniou, P., Knight, S. J. L., Camps, C., Pentony, M. M., Kvikstad, E. M., Harris, S., Popitsch, N., Pagnamenta, A. T., Schuh, A., Taylor, J. C., & Wordsworth, S. (2020). The complete costs

7 Bibliography

- of genome sequencing: a microcosting study in cancer and rare diseases from a single center in the United Kingdom. *Genetics in Medicine*, 22(1), 85–94. <https://doi.org/10.1038/s41436-019-0618-7>
- scikit-learn: machine learning in Python — scikit-learn 1.7.0 documentation.* (n.d.). Retrieved June 30, 2025, from <https://scikit-learn.org/stable/>
- Shao, X., Lv, N., Liao, J., Long, J., Xue, R., Ai, N., Xu, D., & Fan, X. (2019). Copy number variation is highly correlated with differential gene expression: A pan-cancer study. *BMC Medical Genetics*, 20(1), 1–14. <https://doi.org/10.1186/S12881-019-0909-5/FIGURES/6>
- Singh, A. K., Olsen, M. F., Lavik, L. A. S., Vold, T., Drabløs, F., & Sjursen, W. (2021). Detecting copy number variation in next generation sequencing data from diagnostic gene panels. *BMC Medical Genomics*, 14(1), 1–12. <https://doi.org/10.1186/S12920-021-01059-X/TABLES/2>
- Spurr, L. F., Alomran, N., Bousounis, P., Reece-Stremtan, D., Prashant, N. M., Liu, H., Słowiński, P., Li, M., Zhang, Q., Sein, J., Asher, G., Crandall, K. A., Tsaneva-Atanasova, K., & Horvath, A. (2020). ReQTL: identifying correlations between expressed SNVs and gene expression using RNA-sequencing data. *Bioinformatics*, 36(5), 1351–1359. <https://doi.org/10.1093/BIOINFORMATICS/BTZ750>
- Whitford, W., Lehnert, K., Snell, R. G., & Jacobsen, J. C. (2019). Evaluation of the performance of copy number variant prediction tools for the detection of deletions from whole genome sequencing data. *Journal of Biomedical Informatics*, 94, 103174. <https://doi.org/10.1016/J.JBI.2019.103174>
- Xie, X., Xi, X., Zhao, D., Zhao, Y., Yi, T., Chen, D., Liu, R., Qi, L., Pan, Z., Wang, H., Zhang, H., Ding, R., & Du, H. (2024). Advancing pathogen and tumor copy number variation detection through simultaneous metagenomic next-generation sequencing: A comprehensive review. *Heliyon*, 10(21), e38826. <https://doi.org/10.1016/J.HELİYON.2024.E38826>

7 Bibliography

- Zare, F., Dow, M., Monteleone, N., Hosny, A., & Nabavi, S. (2017). An evaluation of copy number variation detection tools for cancer using whole exome sequencing data. *BMC Bioinformatics*, 18(1), 1–13. <https://doi.org/10.1186/S12859-017-1705-X/FIGURES/5>
- Zarrei, M., MacDonald, J. R., Merico, D., & Scherer, S. W. (2015). A copy number variation map of the human genome. *Nature Reviews Genetics* 2015 16:3, 16(3), 172–183. <https://doi.org/10.1038/nrg3871>
- Zhang, Y., Jin, L., Wang, B., Hu, D., Wang, L., Li, P., Zhang, J., Han, K., Tian, G., Yuan, D., Yang, J., Tan, W., Xing, X., & Lang, J. (2019). DL-CNV: A deep learning method for identifying copy number variations based on next generation target sequencing. *Mathematical Biosciences and Engineering*, 17(1), 202–215. <https://doi.org/10.3934/mbe.2020011>